

[Nome do Autor]

**ExoGame: Plataforma Gamificada de Perguntas
e Respostas sobre Exoplanetas com
Sincronização em Tempo Real**

[Cidade]

2026

[Nome do Autor]

ExoGame: Plataforma Gamificada de Perguntas e Respostas sobre Exoplanetas com Sincronização em Tempo Real

Trabalho de Conclusão de Curso apresentado ao Curso de [Nome do Curso] da [Nome da Universidade], como requisito parcial para a obtenção do grau de [Bacharel/Tecnólogo] em [Nome do Curso].

[Nome da Universidade]

[Nome da Faculdade / Departamento]

Curso de [Nome do Curso]

Orientador: [Nome do Orientador]

[Cidade]

2026

[Nome do Autor]

ExoGame: Plataforma Gamificada de Perguntas e Respostas sobre Exoplanetas com Sincronização em Tempo Real/ [Nome do Autor]. – [Cidade], 2026-80p. : il. (algumas color.) ; 30 cm.

Orientador: [Nome do Orientador]

Trabalho de Conclusão de Curso (Graduação) – [Nome da Universidade]

[Nome da Faculdade / Departamento]

Curso de [Nome do Curso], 2026.

1. Gamificação. 2. Exoplanetas. 3. WebSockets. 4. Next.js. 5. Elixir. 6. Phoenix. I. Orientador. II. Universidade. III. Faculdade. IV. Título

[Nome do Autor]

ExoGame: Plataforma Gamificada de Perguntas e Respostas sobre Exoplanetas com Sincronização em Tempo Real

Trabalho de Conclusão de Curso apresentado ao Curso de [Nome do Curso] da [Nome da Universidade], como requisito parcial para a obtenção do grau de [Bacharel/Tecnólogo] em [Nome do Curso].

Trabalho aprovado. [Cidade], ____ de _____ de 2026:

[Nome do Orientador]
Orientador

[Nome do Avaliador 1]
Avaliador 1

[Nome do Avaliador 2]
Avaliador 2

[Cidade]
2026

*Dedico este trabalho a todos aqueles que olham para o céu noturno
e se perguntam se estamos sozinhos no universo.*

Agradecimentos

Agradeço primeiramente a Deus pela oportunidade e força para concluir este trabalho.

Ao meu orientador, por toda a dedicação, paciência e conhecimento compartilhado ao longo desta jornada acadêmica.

À minha família, pelo apoio incondicional e compreensão durante os momentos de dedicação intensa a este projeto.

A todos os professores que contribuíram para a minha formação acadêmica e profissional.

Aos colegas de curso que participaram dos testes da plataforma ExoGame, cujo *feedback* foi essencial para o aprimoramento do sistema.

*“Em algum lugar, algo incrível está esperando
para ser descoberto.”*

— Carl Sagan

Resumo

Este trabalho apresenta o desenvolvimento da plataforma **ExoGame**, um sistema *web* gamificado de perguntas e respostas em tempo real, inspirado na mecânica do Kahoot!, voltado para a divulgação científica e o ensino de conceitos astronômicos, com ênfase em exoplanetas. A plataforma foi construída utilizando exclusivamente **Next.js 15** no *frontend* — com suporte a *Server-Side Rendering* (SSR) e *Turbopack* — e **Phoenix 1.7 (Elixir/BEAM)** no *backend*, com comunicação bidirecional em tempo real viabilizada por **Phoenix Channels** (WebSocket nativo, RFC 6455). A escolha do Elixir fundamenta-se em sua trajetória histórica: criado por José Valim a partir do ecossistema Erlang/OTP — desenvolvido pela Ericsson nos anos 1980 para sistemas de telecomunicação com exigência de tolerância a falhas e alta disponibilidade —, o Elixir herda um modelo de concorrência maduro e comprovado em produção por empresas como o **WhatsApp**, que com Erlang/BEAM atingiu a marca de **2 milhões de conexões simultâneas por servidor**, tornando-se referência mundial em escalabilidade de WebSocket. Nos testes empíricos conduzidos, o Phoenix sustentou **200 conexões WebSocket simultâneas sem erros**, com latência **estável e previsível** sob carga variável, característica do *scheduler* preemptivo da BEAM. O sistema implementa mecânicas de gamificação como pontuação baseada em tempo de resposta, avatares personalizáveis, *leaderboard* dinâmico e sistema de salas com código PIN. Os resultados demonstram que a integração de gamificação com tecnologias *web* modernas constitui uma abordagem promissora para engajar estudantes no aprendizado de conceitos astronômicos complexos.

Palavras-chave: gamificação. exoplanetas. websockets. next.js. elixir. phoenix. beam. quiz interativo. divulgação científica.

Abstract

This work presents the development of the **ExoGame** platform, a gamified real-time web-based question-and-answer system inspired by Kahoot! mechanics, aimed at science communication and teaching astronomical concepts, with emphasis on exoplanets. The platform was built exclusively with **Next.js 15** on the frontend — with Server-Side Rendering (SSR) and Turbopack support — and **Phoenix 1.7 (Elixir/BEAM)** on the backend, with bidirectional real-time communication enabled by **Phoenix Channels** (native WebSocket, RFC 6455). The choice of Elixir is grounded in its history: created by José Valim on top of the Erlang/OTP ecosystem — developed by Ericsson in the 1980s for telecommunication systems requiring fault tolerance and high availability —, Elixir inherits a concurrency model proven in production by companies such as **WhatsApp**, which achieved **2 million simultaneous connections per server** on Erlang/BEAM, establishing it as the world benchmark for WebSocket scalability. Empirical tests showed Phoenix sustaining **200 simultaneous WebSocket connections with zero errors**, with **stable, predictable latency** under varying load due to the BEAM's preemptive scheduler. The system implements gamification mechanics such as time-based scoring, customizable avatars, dynamic leaderboard, and a PIN-based room system. The theoretical framework covers the fields of gamification in education, real-time web technologies, and science communication in Astronomy.

Keywords: gamification. exoplanets. websockets. next.js. elixir. phoenix. beam. interactive quiz. science communication.

Lista de ilustrações

Figura 1 – Arquitetura escalável com Phoenix/PubSub para múltiplos nós Elixir	33
Figura 2 – Arquitetura geral da plataforma ExoGame	49
Figura 3 – Latência média e throughput por nível de concorrência HTTP	62
Figura 4 – Throughput (req/s) por nível de concorrência HTTP	62
Figura 5 – Latência de join no Phoenix Channel por número de conexões WebSocket simultâneas	63
Figura 6 – Diagrama de fluxo de uma partida no ExoGame	77

Lista de quadros

Quadro 1 – Mapeamento das teorias de aprendizagem no ExoGame	23
Quadro 2 – Elementos de gamificação implementados no ExoGame	25
Quadro 3 – Banco de questões sobre exoplanetas implementado no ExoGame . . .	45
Quadro 3 – (continuação) Banco de questões sobre exoplanetas implementado no ExoGame	46
Quadro 4 – Ferramentas e tecnologias utilizadas	47

Lista de tabelas

Tabela 1	–	Evolução da gamificação educacional por plataforma tecnológica	28
Tabela 2	–	Comparativo de estratégias de escalabilidade do backend Phoenix/Elixir	33
Tabela 3	–	Comparativo entre as tecnologias do <i>frontend</i> e <i>backend</i>	36
Tabela 4	–	Linha do tempo dos marcos na história dos exoplanetas	42
Tabela 5	–	Exemplos de transposição de dados NASA para questões do ExoGame	44
Tabela 6	–	Eventos WebSocket da plataforma ExoGame	51
Tabela 7	–	Latência HTTP do endpoint <code>/questions/random</code> por nível de concorrência	59
Tabela 8	–	Comparação de latência entre endpoints HTTP (n=60 reqs, sequencial)	60
Tabela 9	–	Latência de conexão e ingresso (<i>join</i>) no Phoenix Channel por número de conexões simultâneas	61

Lista de abreviaturas e siglas

API	<i>Application Programming Interface</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CSS	<i>Cascading Style Sheets</i>
DTO	<i>Data Transfer Object</i>
HTML	<i>HyperText Markup Language</i>
HTTP	<i>HyperText Transfer Protocol</i>
IoC	<i>Inversion of Control</i>
ISR	<i>Incremental Static Regeneration</i>
JSON	<i>JavaScript Object Notation</i>
JWST	<i>James Webb Space Telescope</i>
JWT	<i>JSON Web Token</i>
MVC	<i>Model-View-Controller</i>
NASA	<i>National Aeronautics and Space Administration</i>
NExScI	<i>NASA Exoplanet Science Institute</i>
PIN	<i>Personal Identification Number</i>
PWA	<i>Progressive Web App</i>
REST	<i>Representational State Transfer</i>
SPA	<i>Single Page Application</i>
SRS	<i>Student Response System</i>
SSR	<i>Server-Side Rendering</i>
TCP	<i>Transmission Control Protocol</i>
TCC	Trabalho de Conclusão de Curso
TESS	<i>Transiting Exoplanet Survey Satellite</i>

TOI	<i>TESS Object of Interest</i>
UA	Unidade Astronômica
UI	<i>User Interface</i>
UX	<i>User Experience</i>
WSRS	<i>Web-based Student Response System</i>

Sumário

1	INTRODUÇÃO	17
1.1	Problema de Pesquisa	17
1.2	Objetivos	18
1.2.1	Objetivo Geral	18
1.2.2	Objetivos Específicos	18
1.3	Justificativa	18
1.4	Estrutura do Trabalho	19
2	FUNDAMENTAÇÃO TEÓRICA	20
2.1	Gamificação no Ensino	20
2.1.1	Conceitos e Definições	20
2.1.2	Fundamentos Teóricos da Aprendizagem	20
2.1.3	Construcionismo de Papert e a Aprendizagem pela Ação	21
2.1.4	Aprendizagem Significativa de Ausubel	22
2.1.5	Síntese: O Aluno como Protagonista Epistêmico	23
2.1.6	Evidências Empíricas	24
2.1.7	Elementos de Gamificação no ExoGame	24
2.1.8	Gamificação e Tecnologia Web: Uma Convergência Transformadora	24
2.1.8.1	Dos Clickers aos Quizzes Web: A Evolução dos Sistemas de Resposta	24
2.1.8.2	Kahoot! como Paradigma da Gamificação Web	26
2.1.8.3	Por Que a Web? Vantagens Técnicas e Pedagógicas	26
2.1.8.4	O Impacto da Pandemia: Aceleração da Convergência	27
2.1.8.5	Síntese: O Papel da Web na Democratização da Gamificação Educacional	28
2.2	Tecnologias Web para Aplicações em Tempo Real	29
2.2.1	A Evolução da Web em Tempo Real	29
2.2.2	Next.js: Framework para o Frontend	29
2.2.3	Elixir: Origem, Filosofia e Trajetória	29
2.2.4	Phoenix: O Framework Web do Elixir	30
2.2.5	Escalabilidade do Phoenix/Elixir: Propriedades do Backend Principal	31
2.2.5.1	Arquitetura Baseada em Processos OTP	31
2.2.5.2	Escalabilidade Horizontal Nativa com Erlang/OTP	32
2.2.5.3	Análise Comparativa de Capacidade	32
2.2.6	WhatsApp e a BEAM: A Prova Industrial da Superioridade Erlang para WebSocket	33
2.2.6.1	A Migração do WhatsApp para Erlang/BEAM	34

2.2.6.2	Marcos de Escalabilidade do WhatsApp com BEAM	34
2.2.6.3	Paralelismo com o ExoGame	34
2.2.7	Comparativo de Tecnologias	35
2.3	Ensino de Astronomia e Divulgação Científica	35
2.3.1	Situação do Ensino de Astronomia no Brasil	35
2.3.2	Exoplanetas: Uma Trajetória Histórica de Quatro Séculos	36
2.3.2.1	Os Precursores Filosóficos: De Epicuro a Giordano Bruno	36
2.3.2.2	O Período Newtoniano e os Primeiros Cálculos	37
2.3.2.3	Século XIX: Primeiras Tentativas de Detecção	37
2.3.2.4	A Proposta de Otto Struve e os Métodos Indiretos	38
2.3.2.5	A Primeira Detecção: Planetas ao Redor de um Pulsar (1992)	38
2.3.2.6	A Revolução de 1995: 51 Pegasi b e o Júpiter Quente	39
2.3.2.7	A Era Kepler: A Explosão Estatística (2009–2018)	39
2.3.2.8	TESS e a Era Atual (2018–Presente)	40
2.3.2.9	Linha do Tempo: Marcos na História dos Exoplanetas	41
2.3.2.10	Significado para a Educação Científica	41
2.3.3	Acurácia Científica: O NASA Exoplanet Archive como Fonte Primária	41
2.3.3.1	Caracterização do NASA Exoplanet Archive	41
2.3.3.2	Protocolo de Elaboração de Questões	43
2.3.3.3	Validade Científica como Diferencial Pedagógico	43
2.3.3.4	Banco de Questões Implementado	44
2.3.4	Tecnologias Interativas na Divulgação Científica	44
3	METODOLOGIA	47
3.1	Classificação da Pesquisa	47
3.2	Ferramentas de Desenvolvimento	47
3.3	Processo de Desenvolvimento	48
4	ARQUITETURA DO SISTEMA	49
4.1	Visão Geral da Arquitetura	49
4.2	Estrutura de Módulos do <i>Backend</i>	49
4.3	Estrutura de Componentes do <i>Frontend</i>	50
4.4	Protocolo de Comunicação WebSocket	50
5	DETALHES DE IMPLEMENTAÇÃO	52
5.1	Interfaces de Dados	52
5.2	Gateway WebSocket: Phoenix Channels	53
5.3	Sistema de Pontuação	53
5.4	Gerenciamento de Estado no <i>Frontend</i>	54
5.5	Validação de Dados via <i>Pattern Matching</i>	55

6	RESULTADOS E DISCUSSÃO	57
6.1	Funcionalidades Implementadas	57
6.1.1	Fluxo de Criação de Sala	57
6.1.2	Fluxo de Entrada via PIN	57
6.1.3	Dinâmica do Quiz	58
6.1.4	Sistema de Pontuação	58
6.2	Segurança nas Comunicações	58
6.3	Avaliação Empírica de Desempenho do <i>Backend</i>	59
6.3.1	Metodologia dos Testes	59
6.3.2	Latência HTTP por Nível de Concorrência	59
6.3.3	Comparação entre Endpoints HTTP	60
6.3.4	Throughput Sustentado	60
6.3.5	Conexões WebSocket Simultâneas (Phoenix Channels)	61
6.3.6	Visualização Gráfica dos Resultados	61
6.3.7	Interpretação dos Resultados	61
6.4	Discussão	63
7	TRABALHOS FUTUROS	64
7.1	Visualização 3D de Exoplanetas com Three.js	64
7.2	Algoritmos de Recomendação para Trilhas de Aprendizagem	64
7.2.1	Modelo de Desempenho do Aluno	65
7.2.2	Algoritmo de Recomendação	65
7.3	Outras Melhorias Propostas	65
8	CONCLUSÃO	67
	REFERÊNCIAS	69
	APÊNDICES	73
	APÊNDICE A – ESTRUTURA DE DIRETÓRIOS DO PROJETO	74
	APÊNDICE B – DIAGRAMA DE FLUXO DA PARTIDA	76
	APÊNDICE C – CONFIGURAÇÃO DOCKER COMPOSE	78

1 Introdução

A descoberta de exoplanetas — planetas que orbitam estrelas além do nosso Sol — representa uma das áreas mais fascinantes e dinâmicas da astronomia contemporânea. Desde a detecção pioneira do 51 Pegasi b por [Mayor e Queloz \(1995\)](#), mais de cinco mil exoplanetas foram confirmados, muitos deles em zonas habitáveis de suas respectivas estrelas ([NASA Exoplanet Archive, 2024](#)). Contudo, apesar da riqueza de descobertas recentes, a divulgação desses conhecimentos para o grande público e, em particular, para estudantes do ensino básico e superior, permanece aquém do desejado ([LANGHI; NARDI, 2009](#)).

Paralelamente, a revolução digital tem proporcionado o surgimento de ferramentas educacionais inovadoras que combinam tecnologia e pedagogia de formas cada vez mais sofisticadas. A **gamificação** — definida por [Deterding et al. \(2011\)](#) como o uso de elementos de *design* de jogos em contextos não-lúdicos — tem se consolidado como uma estratégia eficaz para aumentar o engajamento e a motivação dos aprendizes ([HAMARI; KOIVISTO; SARSA, 2014](#); [KAPP, 2012](#)). Plataformas como o Kahoot! demonstraram o potencial de quizzes interativos em tempo real para transformar a dinâmica de sala de aula ([WANG; TAHIR, 2020](#); [LICORISH et al., 2018](#)).

Nesse cenário, as tecnologias *web* modernas oferecem os alicerces técnicos para a construção de plataformas interativas e responsivas. *Frameworks* como o **Next.js** para o *frontend* e o **Phoenix** (Elixir/BEAM) para o *backend*, aliados ao protocolo **WebSocket** para comunicação bidirecional em tempo real, possibilitam a criação de experiências de aprendizagem dinâmicas e colaborativas, acessíveis por meio de qualquer navegador *web*.

1.1 Problema de Pesquisa

A divulgação de conceitos astronômicos complexos, como os relacionados a exoplanetas, enfrenta desafios significativos. A abstração inerente ao tema, a distância entre o fenômeno estudado e a experiência cotidiana do aluno, e a ausência de ferramentas interativas específicas para o ensino de Astronomia são barreiras recorrentes ([LANGHI; NARDI, 2009](#); [BRETONES; COMPIANI, 2006](#)). Diante disso, formula-se a seguinte questão norteadora: **Como uma plataforma *web* gamificada, com mecânicas de quiz em tempo real, pode contribuir para a divulgação científica e o engajamento de estudantes no aprendizado sobre exoplanetas?**

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver e apresentar a plataforma ExoGame, um sistema *web* gamificado de perguntas e respostas em tempo real sobre exoplanetas, utilizando exclusivamente Next.js no *frontend* e Phoenix/Elixir no *backend*, com comunicação via Phoenix Channels (WebSocket nativo), e avaliar empiricamente o desempenho da solução.

1.2.2 Objetivos Específicos

- Realizar uma revisão bibliográfica sobre gamificação no ensino de ciências, tecnologias *web* em tempo real e divulgação científica em Astronomia;
- Projetar e implementar uma arquitetura cliente-servidor moderna empregando exclusivamente Next.js 15 e **Phoenix 1.7 (Elixir/BEAM)** com WebSocket via Phoenix Channels;
- Contextualizar historicamente a linguagem Elixir e o ecossistema Erlang/OTP do qual deriva, situando a escolha tecnológica dentro de uma trajetória consolidada de robustez e concorrência;
- Avaliar empiricamente o desempenho do *backend* Phoenix/Elixir por meio de testes de carga, e argumentar pela adequação do Phoenix/BEAM para aplicações WebSocket de alta concorrência, com suporte em casos reais de produção como o WhatsApp;
- Implementar mecânicas de gamificação como pontuação baseada em tempo de resposta, avatares personalizáveis e *leaderboard* dinâmico;
- Documentar a arquitetura técnica e os fluxos de interação do sistema;
- Propor extensões futuras, incluindo visualização 3D de exoplanetas e algoritmos de recomendação para trilhas de aprendizagem.

1.3 Justificativa

A relevância deste trabalho reside na intersecção de três domínios estratégicos: (i) a necessidade de ferramentas inovadoras para a divulgação científica em Astronomia; (ii) a eficácia comprovada da gamificação como estratégia de engajamento educacional; e (iii) a maturidade das tecnologias *web* para suportar aplicações em tempo real com baixa latência.

Estudos como o de Wang e Tahir (2020) demonstram que quizzes interativos em tempo real melhoram significativamente a atenção e a retenção de conteúdo por parte

dos alunos. Ao direcionar essa abordagem para o ensino de Astronomia, especificamente exoplanetas, busca-se preencher uma lacuna na oferta de ferramentas interativas para esta área do conhecimento.

1.4 Estrutura do Trabalho

Este trabalho está organizado da seguinte forma: o [Capítulo 2](#) apresenta a fundamentação teórica, abrangendo gamificação, tecnologias *web* e ensino de Astronomia; o [Capítulo 3](#) descreve a metodologia de desenvolvimento adotada; o [Capítulo 4](#) detalha a arquitetura do sistema ExoGame; o [Capítulo 5](#) apresenta os detalhes de implementação; o [Capítulo 6](#) discute os resultados obtidos e as funcionalidades implementadas; o [Capítulo 7](#) propõe trabalhos futuros; e o [Capítulo 8](#) encerra com as considerações finais.

2 Fundamentação Teórica

Este capítulo apresenta os fundamentos teóricos que sustentam o desenvolvimento da plataforma ExoGame. A revisão está organizada em três eixos temáticos: gamificação e teorias de aprendizagem, tecnologias *web* para aplicações em tempo real, e divulgação científica no ensino de Astronomia.

2.1 Gamificação no Ensino

2.1.1 Conceitos e Definições

O termo **gamificação** (do inglês, *gamification*) foi definido por [Deterding et al. \(2011\)](#) como “o uso de elementos de *design* de jogos em contextos não relacionados a jogos”. Essa definição enfatiza a transposição seletiva de mecânicas de jogos — tais como pontuação, *rankings*, desafios e recompensas — para domínios como educação, saúde e negócios, sem necessariamente criar um jogo completo.

[Kapp \(2012\)](#) amplia essa perspectiva ao definir gamificação como “o uso de mecânicas baseadas em jogos, estética e pensamento lúdico para engajar pessoas, motivar ações, promover aprendizagem e resolver problemas”. Para Kapp, a gamificação não se limita à adição superficial de pontos e *badges*, mas envolve o uso de narrativa, desafio, cooperação/competição e *feedback* imediato como elementos centrais de design.

2.1.2 Fundamentos Teóricos da Aprendizagem

A eficácia da gamificação no ensino pode ser compreendida à luz de diversas teorias pedagógicas e psicológicas:

- **Teoria da Autodeterminação** ([RYAN; DECI, 2000](#)): propõe que a motivação intrínseca é sustentada pela satisfação de três necessidades psicológicas básicas: *autonomia*, *competência* e *pertencimento*. Mecânicas de gamificação como escolha de avatares (autonomia), pontuação e *leaderboard* (competência) e jogo em grupo (pertencimento) alinham-se diretamente a essas necessidades;
- **Teoria do Fluxo** ([CSIKSZENTMIHALYI, 1990](#)): descreve o estado de “fluxo” como uma experiência de imersão completa em uma atividade, atingido quando há equilíbrio entre o nível de desafio e a habilidade do indivíduo. Quizzes com temporizador criam esse equilíbrio ao oferecer pressão temporal controlada;

- **Construtivismo Social** (VYGOTSKY, 1978): enfatiza que a aprendizagem ocorre na interação social. O formato de quiz em grupo, com discussão de resultados e *ranking*, promove a construção coletiva de conhecimento.

2.1.3 Construcionismo de Papert e a Aprendizagem pela Ação

Seymour Papert, discípulo de Piaget e cofundador do MIT Media Lab, propôs o **Construcionismo** como uma extensão radical do construtivismo piagetiano. Enquanto Piaget descreveu como a criança constrói estruturas cognitivas internamente, Papert (PAPERT, 1980; PAPERT, 1993) argumentou que essa construção é potencializada quando o aprendiz está engajado na *construção de um artefato externo* — um programa, um robô, uma simulação — que seja pessoalmente significativo.

O princípio central do Construcionismo pode ser sintetizado na máxima: “*learning by making*”. Para Papert, o computador não deveria ser usado para “programar a criança”, mas sim como um instrumento para que “a criança programe o computador”, invertendo a relação passiva tradicionalmente imposta pelas tecnologias educacionais (PAPERT, 1980).

No contexto da plataforma ExoGame, o Construcionismo se manifesta em múltiplas camadas:

1. **O aluno como participante ativo:** diferentemente de uma aula expositiva sobre exoplanetas, na qual o aluno é espectador passivo, o quiz interativo exige tomada de decisão constante — o aluno deve mobilizar seus conhecimentos, avaliar alternativas e agir sob pressão temporal. Essa dinâmica transforma o aprendiz de *receptor de informações* em *agente epistêmico*;
2. **Feedback como motor de construção:** cada resposta no ExoGame gera um *feedback* imediato — o campo `correctAnswerContext` oferece uma explicação contextualizada após cada questão, permitindo que o aluno reconstrua seus modelos mentais em tempo real. Papert (PAPERT, 1993) denominava esse ciclo de “*debugging*” cognitivo: o erro não é punição, mas oportunidade de revisão;
3. **Micromundos interativos:** Papert defendia a criação de “micromundos” — ambientes simplificados nos quais conceitos complexos podem ser explorados experimentalmente. O ExoGame funciona como um *micromundo astronômico*, onde conceitos como zona habitável, métodos de detecção e tipos planetários são apresentados em doses gerenciáveis de 15 a 20 segundos, respeitando a capacidade atencional do aprendiz;
4. **Dimensão social da construção:** o formato multiplayer do quiz adiciona a camada social que Papert reconhecia como catalisadora da aprendizagem. A discussão pós-

resultado entre jogadores — “por que a resposta era B e não C?” — constitui um processo de negociação de significados que aprofunda a compreensão conceitual.

2.1.4 Aprendizagem Significativa de Ausubel

David Ausubel (AUSUBEL, 1968; AUSUBEL, 2000) propôs a teoria da **Aprendizagem Significativa** (*Meaningful Learning*), segundo a qual a aprendizagem ocorre quando novas informações são ancoradas a **subsunçores** — conceitos já existentes na estrutura cognitiva do aprendiz. Ausubel a contrapôs à *aprendizagem mecânica* (*rote learning*), na qual a informação é memorizada de forma literal e arbitrária, sem conexão com o conhecimento prévio.

Para que a aprendizagem seja significativa, Ausubel estabeleceu duas condições necessárias:

1. **Material potencialmente significativo:** o conteúdo deve possuir significado lógico (ser coerente e não trivial) e deve ser *relacionável* à estrutura cognitiva do aprendiz de modo não arbitrário;
2. **Disposição para aprender:** o aprendiz deve apresentar predisposição ativa para relacionar o novo material aos seus subsunçores, rejeitando a memorização mecânica.

A plataforma ExoGame operacionaliza ambas as condições de Ausubel de forma deliberada:

- **Organizadores prévios:** as perguntas do quiz funcionam como **organizadores prévios comparativos** (AUSUBEL, 1968), que mobilizam o conhecimento existente do aluno antes de apresentar a informação correta. Ao ler “Qual método é mais utilizado para detectar exoplanetas?” e ponderar entre as alternativas, o aluno é forçado a ativar seus subsunçores sobre Astronomia, criando “ganchos cognitivos” para a informação que será revelada no *feedback*;
- **Diferenciação progressiva:** o sequenciamento das questões — partindo de conceitos mais gerais (“O que é um exoplaneta?”) para conceitos mais específicos (“Qual exoplaneta na zona habitável de TRAPPIST-1 possui maior probabilidade de conter água líquida?”) — segue o princípio ausubeliano de **diferenciação progressiva**, no qual conceitos gerais e inclusivos são apresentados primeiro e progressivamente diferenciados;
- **Reconciliação integrativa:** o campo `correctAnswerContext` promove a **reconciliação integrativa** ao explicitar relações entre conceitos que o aluno pode não ter percebido. Por exemplo, ao explicar que “Plutão foi reclassificado como planeta anão

em 2006 pela União Astronômica Internacional”, o sistema conecta a questão sobre planetas do Sistema Solar com a definição formal de planeta, integrando conceitos de classificação astronômica;

- **Motivação como pré-condição:** a gamificação (pontuação, *leaderboard*, competição social) atuam diretamente sobre a segunda condição de Ausubel — a *disposição para aprender*. A mecânica competitiva transforma o que seria uma lista de perguntas inerte em uma experiência intrinsecamente motivadora, na qual o aluno *quer* processar a informação semanticamente (e não mecanicamente) porque busca a resposta correta com convicção.

2.1.5 Síntese: O Aluno como Protagonista Epistêmico

A convergência entre o Construcionismo de Papert e a Aprendizagem Significativa de Ausubel no design do ExoGame pode ser visualizada no [Quadro 1](#).

Quadro 1 – Mapeamento das teorias de aprendizagem no ExoGame

Princípio Teórico	Autor	Manifestação no ExoGame	Componente Técnico
Aprender fazendo	Papert	Tomada de decisão ativa sob pressão temporal	QuestionView + timer
<i>Debugging</i> cognitivo	Papert	Feedback contextualizado após cada questão	correctAnswerContext
Micromundo	Papert	Conceitos astronômicos em doses de 15–20s	timeLimit por questão
Subsunçor	Ausubel	Ativação de conhecimento prévio pelas alternativas	Opções do quiz
Diferenciação progressiva	Ausubel	Sequenciamento de questões simples para complexas	getRandomQuestions()
Reconciliação integrativa	Ausubel	Explicação conectando conceitos no feedback	correctAnswerContext
Disposição para aprender	Ausubel	Gamificação como motivadora do processamento semântico	Pontuação + leaderboard

Fonte: Elaborado pelo autor.

Em síntese, o ExoGame materializa a transição do aluno de **espectador passivo** para **participante ativo** — ou, nos termos de Papert, de “programado” para “programa-

dor” de sua própria aprendizagem. Essa transformação é viabilizada pela conjunção de três fatores: **(i)** a mecânica de quiz que demanda ação constante; **(ii)** o *feedback* contextualizado que ancora novos conceitos a subsunções existentes; e **(iii)** a gamificação que sustenta a disposição motivacional necessária para a aprendizagem significativa.

2.1.6 Evidências Empíricas

A literatura oferece evidências consistentes sobre o impacto positivo da gamificação na educação. A metanálise de Hamari, Koivisto e Sarsa (2014), que revisou 24 estudos empíricos, concluiu que a gamificação produz efeitos positivos, embora ressalte que os resultados dependem do contexto e dos elementos de *design* empregados.

Sailer et al. (2017) conduziram um estudo experimental com 419 participantes, demonstrando que diferentes elementos de gamificação atendem a diferentes necessidades psicológicas. Especificamente, pontos, *leaderboards* e badges tiveram efeito significativo na satisfação da necessidade de competência.

No contexto de plataformas de quiz interativo, Wang e Tahir (2020) realizaram uma revisão de 93 estudos sobre o uso do Kahoot! em sala de aula. Os resultados indicam impactos positivos na motivação, engajamento, concentração e aprendizagem percebida dos estudantes.

2.1.7 Elementos de Gamificação no ExoGame

O Quadro 2 apresenta os elementos de gamificação implementados na plataforma ExoGame e sua correspondência com os fundamentos teóricos.

2.1.8 Gamificação e Tecnologia Web: Uma Convergência Transformadora

A interseção entre gamificação e tecnologia *web* representa uma das convergências mais significativas na educação contemporânea. Esta subseção analisa como a evolução da *web* não apenas viabilizou a gamificação em escala, mas transformou qualitativamente as possibilidades pedagógicas, criando um novo paradigma de ensino interativo, acessível e baseado em dados.

2.1.8.1 Dos Clickers aos Quizzes Web: A Evolução dos Sistemas de Resposta

A pré-história da gamificação educacional digital pode ser traçada aos **Sistemas de Resposta do Estudante** (*Student Response Systems* — SRS), popularmente conhecidos como *clickers*. Surgidos na década de 1990, os *clickers* eram dispositivos de hardware proprietário que permitiam aos alunos responder perguntas de múltipla escolha em sala de aula, com os resultados sendo agregados e exibidos em tempo real (CALDWELL, 2007).

Quadro 2 – Elementos de gamificação implementados no ExoGame

Elemento	Implementação	Base Teórica
Pontuação	Cálculo baseado em acerto e tempo de resposta ($1000 + \text{bônus}_{\text{tempo}} \times 10$)	Competência (RYAN; DECI, 2000)
<i>Leaderboard</i>	Ranking dinâmico atualizado em tempo real após cada questão	Competência e pertencimento (SAILER et al., 2017)
Avatares	Seleção de 32 avatares (emojis) com identificação visual	Autonomia (RYAN; DECI, 2000)
Temporização	Contagem regressiva com barra de progresso visual (15–20s)	Fluxo (CSIKSZENT-MIHALYI, 1990)
<i>Feedback</i> imediato	Confirmação de resposta e revelação do resultado correto	Construtivismo (PIAGET, 1964)
Competição social	Jogo em tempo real com múltiplos participantes simultâneos	Pertencimento (VYGOTSKY, 1978)

Fonte: Elaborado pelo autor.

Embora os *clickers* tenham demonstrado impactos positivos no engajamento e na participação dos alunos, apresentavam limitações estruturais significativas:

- **Custo:** cada dispositivo tinha custo unitário elevado, e as instituições precisavam adquirir frotas de centenas de unidades;
- **Logística:** distribuição, coleta e manutenção dos dispositivos antes e depois de cada aula;
- **Funcionalidade limitada:** tela monocromática, sem suporte a imagens, vídeos ou interações complexas;
- **Dependência de fabricante:** cada sistema era proprietário e incompatível com outros.

A transição para SRS baseados em *web* — comumente designados **WSRS** (*Web-based Student Response Systems*) — eliminou essas barreiras ao transformar o próprio dispositivo pessoal do aluno (smartphone, tablet, laptop) no “*clicker*”. A *web* como plataforma garantiu: **custo zero** de hardware adicional, **acesso universal** via navegador, **interfaces ricas** com multimídia, e **atualização contínua** sem necessidade de substituição de dispositivos (WANG, 2015).

2.1.8.2 Kahoot! como Paradigma da Gamificação Web

O **Kahoot!**, lançado em 2013 na Noruega por Johan Brand, Jamie Brooker e Morten Versvik em parceria com a Universidade Norueguesa de Ciência e Tecnologia (NTNU), tornou-se o exemplo paradigmático de como a tecnologia *web* pode potencializar a gamificação educacional em escala global (WANG; TAHIR, 2020).

A arquitetura do Kahoot! é deceptivamente simples: um professor cria um quiz, projeta as perguntas na tela da sala, e os alunos respondem em seus dispositivos pessoais via navegador *web*, identificados por um código PIN numérico. Contudo, por trás dessa simplicidade aparente, o Kahoot! incorpora múltiplos princípios de gamificação:

1. **Pontuação baseada em velocidade:** recompensa não apenas o acerto, mas a rapidez da resposta, ativando o sistema dopaminérgico de recompensa (RYAN; DECI, 2000);
2. **Leaderboard em tempo real:** após cada pergunta, um ranking é exibido, criando ciclos de competição social que sustentam o engajamento;
3. **Música e efeitos visuais:** o design audiovisual do Kahoot! é deliberadamente projetado para evocar a estética de programas de competição televisiva, criando uma atmosfera de “jogo” que rompe com a formalidade da sala de aula;
4. **Acessibilidade web:** ao funcionar inteiramente no navegador, sem necessidade de instalação de aplicativos ou *plugins*, o Kahoot! democratizou o acesso à gamificação educacional.

A revisão sistemática de Wang e Tahir (2020), que analisou 93 estudos sobre o uso do Kahoot! na educação, identificou impactos positivos consistentes em cinco dimensões: (i) desempenho acadêmico, (ii) dinâmica de sala de aula, (iii) atitude dos estudantes, (iv) ansiedade (redução) e (v) percepção de aprendizagem. Os autores concluíram que o formato de quiz *web* em tempo real é particularmente eficaz porque combina três gatilhos motivacionais: a competição social, o *feedback* imediato e a baixa barreira de entrada tecnológica.

2.1.8.3 Por Que a Web? Vantagens Técnicas e Pedagógicas

A escolha da plataforma *web* como meio para a gamificação educacional não é arbitrária. As tecnologias *web* modernas oferecem vantagens estruturais que amplificam os efeitos da gamificação:

1. **Acessibilidade universal:** aplicações *web* funcionam em qualquer dispositivo com navegador — computadores, smartphones, tablets — sem necessidade de instala-

ção. Isso elimina barreiras socioeconômicas: escolas com poucos recursos podem implementar gamificação sem investimento em hardware ou software proprietário;

2. **Tempo real via WebSocket:** o protocolo WebSocket, padronizado pela RFC 6455, permite comunicação bidirecional e *full-duplex* entre cliente e servidor (LUBBERS; GRECO, 2010). Para a gamificação, isso significa que pontuações, *rankings* e estados de jogo são sincronizados entre todos os participantes com latência inferior a 100ms — condição essencial para a experiência de “jogo ao vivo” que sustenta o engajamento;
3. **Escalabilidade:** diferentemente de aplicações *desktop*, que exigem instalação individual, uma aplicação *web* pode atender milhares de usuários simultâneos a partir de uma única implantação. Isso permite que um professor aplique o mesmo quiz para 30 alunos em sala de aula ou para 3.000 alunos em um evento remoto, sem modificação técnica;
4. **Coleta de dados para análise:** aplicações *web* permitem o registro detalhado de interações — tempo de resposta, padrão de erros, taxa de desistência, progressão ao longo do tempo. Esses dados podem alimentar *dashboards* para professores e, em implementações avançadas, sistemas de aprendizagem adaptativa (BRUSILOVSKY; PEYLO, 2003);
5. **Iteração rápida:** *frameworks* modernos como Next.js (React) e Phoenix (Elixir/BEAM) permitem ciclos de desenvolvimento acelerados com recompilação automática e ecossistemas maduros de bibliotecas. O modo `mix phx.server` do Phoenix (com *code reloading* automático em desenvolvimento) e o Turbopack do Next.js facilitam a experimentação com diferentes mecânicas de gamificação e sua validação empírica;
6. **Progressividade:** tecnologias como *Service Workers* e *Progressive Web Apps* (PWA) permitem que aplicações *web* funcionem *offline* e sejam instaláveis na tela inicial do dispositivo, aproximando-se da experiência de aplicativos nativos sem os custos de desenvolvimento e distribuição em lojas de aplicativos (MAJCHRZAK; BIØRN-HANSEN; GRØNLI, 2018).

2.1.8.4 O Impacto da Pandemia: Aceleração da Convergência

A pandemia de COVID-19 (2020–2021) atuou como catalisador involuntário da convergência entre gamificação e tecnologias *web* na educação. Com o fechamento de escolas e a migração massiva para o ensino remoto, educadores em todo o mundo buscaram ferramentas *web* que pudessem replicar o engajamento da sala de aula presencial em ambientes virtuais (DICHEVA et al., 2015).

Plataformas de quiz gamificado experimentaram crescimento exponencial: o Kahoot! reportou um aumento de 200% no número de sessões entre 2019 e 2020. Simultaneamente, surgiram novas plataformas (Quizizz, Blooket, Gimkit) que exploraram mecânicas de gamificação ainda mais sofisticadas — sistemas de *loot*, narrativas progressivas, personalização de avatares — todas construídas sobre a mesma base tecnológica *web*.

Esse contexto reforça a relevância da plataforma ExoGame: desenvolvida como aplicação *web* moderna com Next.js e Phoenix/Elixir (BEAM), ela herda todas as vantagens de acessibilidade e escalabilidade da *web*, ao mesmo tempo em que direciona a gamificação para um nicho específico — o ensino de Astronomia e exoplanetas — que carece de ferramentas interativas dedicadas.

2.1.8.5 Síntese: O Papel da *Web* na Democratização da Gamificação Educacional

A [Tabela 1](#) sintetiza a evolução histórica da relação entre gamificação educacional e tecnologia.

Tabela 1 – Evolução da gamificação educacional por plataforma tecnológica

Período	Tecnologia	Exemplo	Modelo de interação	Limitação principal
1990–2005	<i>Clickers</i> (hardware)	TurningPoint, iClicker	Resposta individual, resultado agregado	Custo, logística
2005–2012	<i>Web</i> 2.0 (AJAX)	Poll Everywhere	Resposta via SMS/web, resultado projetado	Sem gamificação, baixa interatividade
2013–2019	<i>Web</i> + Web-Socket	Kahoot!, Quizizz	Quiz em tempo real, <i>leaderboard</i> , avatares	Genérico, sem foco disciplinar
2020–atual	<i>Web</i> moderna (React, SSR)	Blooket, Gimkit, ExoGame	Mecânicas avançadas, adaptatividade, PWA	Em desenvolvimento

Fonte: Elaborado pelo autor.

Em conclusão, a tecnologia *web* não é um mero veículo neutro para a gamificação — ela é um **fator constitutivo** que expande as possibilidades pedagógicas. A comunicação bidirecional em tempo real, a acessibilidade universal, a capacidade de coleta de dados e a iteração rápida de desenvolvimento transformaram a gamificação de uma estratégia artesanal (possível apenas com hardware dedicado) em uma abordagem escalável, inclusiva e cientificamente mensurável. A plataforma ExoGame, ao combinar essas vantagens com

o foco em divulgação científica sobre exoplanetas, posiciona-se nessa convergência como contribuição tanto técnica quanto pedagógica.

2.2 Tecnologias *Web* para Aplicações em Tempo Real

2.2.1 A Evolução da *Web* em Tempo Real

A *web* tradicional baseia-se no modelo de requisição-resposta do protocolo HTTP, no qual o cliente inicia toda comunicação. Essa limitação tornou-se um obstáculo para aplicações interativas que demandam atualização instantânea de dados, como jogos multiplayer, chats e sistemas colaborativos.

O protocolo **WebSocket**, padronizado pela RFC 6455, superou essa limitação ao estabelecer um canal de comunicação bidirecional e *full-duplex* sobre uma única conexão TCP. Diferentemente de técnicas anteriores como *long polling* e *server-sent events*, o WebSocket permite que tanto o cliente quanto o servidor enviem mensagens a qualquer momento, com latência mínima (LUBBERS; GRECO, 2010; PURANIK; JOSHI, 2024).

2.2.2 Next.js: *Framework* para o *Frontend*

O **Next.js** é um *framework* React criado pela Vercel que oferece funcionalidades avançadas de renderização (Vercel Inc., 2024):

- **Server-Side Rendering (SSR)**: renderiza páginas no servidor, melhorando o tempo de carregamento inicial e o SEO;
- **Incremental Static Regeneration (ISR)**: revalidação incremental de páginas estáticas;
- **Turbopack**: bundler incremental escrito em Rust, substituto do Webpack, com compilação significativamente mais rápida;
- **App Router**: sistema de roteamento baseado em diretórios com suporte a *layouts* compartilhados e *React Server Components*.

2.2.3 Elixir: Origem, Filosofia e Trajetória

O **Elixir** nasceu da experiência prática de **José Valim**, desenvolvedor brasileiro que, entre 2010 e 2011, integrava a equipe central do *framework* Ruby on Rails. Ao investigar formas de melhorar o desempenho e a concorrência do Rails em cenários de alto volume de requisições, Valim esbarrou nas limitações estruturais do Ruby — em particular no *Global Interpreter Lock* (GIL), que impede a execução verdadeiramente paralela de

threads (VALIM; Elixir Core Team, 2024). Essa investigação o levou a estudar o **Erlang**, uma linguagem funcional criada por **Joe Armstrong**, Robert Virding e Mike Williams na **Ericsson**, em meados da década de 1980, para resolver um problema muito específico e exigente: comutar chamadas telefônicas com disponibilidade próxima de 100% e sem interromper o serviço mesmo diante de falhas de *hardware* ou *software* (ARMSTRONG, 2003). O resultado desse esforço foi o switch AXD301, que atingiu a lendária marca de **nove noves de disponibilidade** (99,9999999%), o equivalente a menos de 32 milissegundos de indisponibilidade por ano (ARMSTRONG, 2013).

Reconhecendo que o modelo de concorrência e tolerância a falhas do Erlang/OTP (*Open Telecom Platform*) era exatamente o que faltava ao ecossistema Ruby, José Valim decidiu não modificar o Ruby, mas criar uma **nova linguagem** que rodasse sobre a própria Máquina Virtual Erlang (BEAM), unindo a robustez comprovada do Erlang a uma sintaxe moderna, legível e produtiva, inspirada em linguagens como Ruby. O projeto Elixir foi anunciado publicamente em **janeiro de 2012** e alcançou a versão 1.0 em **setembro de 2014**, marco que sinalizou a maturidade da linguagem para uso em produção (VALIM; Elixir Core Team, 2024).

A partir daí, o ecossistema Elixir cresceu de forma consistente. Em 2014, **Chris McCord** criou o **Phoenix**, o *framework web* que se tornaria o principal ponto de entrada para desenvolvedores que buscam produtividade e alta concorrência simultaneamente; em 2019, o mesmo autor lançou o **Phoenix LiveView**, permitindo interfaces *web* ricas e reativas em tempo real com o mínimo de JavaScript. O modelo de concorrência do Erlang/BEAM — validado industrialmente por décadas em centrais telefônicas — passou a sustentar sistemas de mensageria e comunicação em tempo real de grande escala, do qual o caso do **WhatsApp** (detalhado na subseção 2.2.6) é o exemplo mais emblemático. Empresas como **Discord**, **Pinterest**, **PepsiCo**, **Financial Times** e **Bleacher Report** adotaram Elixir e Phoenix em sistemas de produção que demandam alta concorrência e disponibilidade (MCCORD; Phoenix Core Team, 2024).

Essa trajetória — de uma investigação sobre os limites de concorrência do Ruby, passando pela redescoberta de uma tecnologia de telecomunicações com décadas de robustez comprovada, até a consolidação de um ecossistema *web* moderno e produtivo — é o que fundamenta a escolha do Elixir como linguagem única do *backend* da plataforma ExoGame. Trata-se de uma linguagem jovem no universo *web*, mas construída sobre uma base de concorrência e tolerância a falhas com décadas de validação em ambientes de missão crítica.

2.2.4 Phoenix: O *Framework Web* do Elixir

O **Phoenix** é o principal *framework web* para Elixir, projetado para alta concorrência e baixa latência (MCCORD; Phoenix Core Team, 2024). É o *framework* adotado como *backend* da plataforma ExoGame, escolha que se fundamenta nas propriedades da BEAM

para aplicações WebSocket de alto volume — discutidas em detalhe nas próximas subseções e referendadas pelos resultados empíricos do [Capítulo 6](#). Suas principais características são:

- **Canais (*Phoenix Channels*)**: abstração nativa para comunicação bidirecional via WebSocket, com suporte a *topics* e *Phoenix.Presence*;
- **Concorrência via processos leves**: cada conexão WebSocket é um processo Erlang isolado, com custo de memória de $\sim 2\text{KB}$ por processo;
- **Tolerância a falhas (OTP)**: supervisores reiniciam automaticamente processos com falha, garantindo alta disponibilidade sem intervenção manual;
- ***Pattern matching* e funções puras**: código funcional, expressivo e de fácil manutenção;
- **PubSub nativo**: o módulo `Phoenix.PubSub` permite *broadcast* eficiente de mensagens entre processos e nós distribuídos sem configuração adicional.

2.2.5 Escalabilidade do Phoenix/Elixir: Propriedades do *Backend* Principal

Esta subseção descreve as propriedades de escalabilidade do *backend* Phoenix/Elixir da plataforma. Os resultados empíricos dos testes de desempenho são apresentados no [Capítulo 6](#).

2.2.5.1 Arquitetura Baseada em Processos OTP

O modelo de execução do Elixir/Erlang é intrinsecamente concorrente, baseado no **Modelo de Atores** (*Actor Model*) da Máquina Virtual BEAM ([ARMSTRONG, 2013](#)). Na plataforma ExoGame, cada sala de jogo é gerenciada como um processo `GenServer` isolado, supervisionado pela árvore de supervisores OTP. Os Canais Phoenix (*Phoenix Channels*) associam cada conexão WebSocket a um processo leve dedicado, garantindo isolamento total entre salas e tolerância a falhas nativa.

O fluxo de eventos opera da seguinte forma:

1. Quando um jogador emite `submit_answer`, o `GameChannel` recebe o evento como uma mensagem Elixir tratada pela função `handle_in/3`;
2. O `GameServer` (`GenServer`) processa a resposta (validação, cálculo de pontuação) de forma isolada em seu estado interno — operação com complexidade $O(1)$ via `Map` Elixir;

3. O canal realiza *broadcast* de volta apenas para o *topic* correspondente (`broadcast!(socket, "answer_stats_updated", payload)`);
4. O `Phoenix.PubSub` distribui o *broadcast* eficientemente para todos os processos assinantes daquele tópico, sem estrutura intermediária.

Essa arquitetura permite que milhares de salas coexistam no mesmo nó sem interferência mútua, pois cada processo `GenServer` opera sobre seu próprio estado isolado e uma falha em um processo não afeta os demais.

2.2.5.2 Escalabilidade Horizontal Nativa com Erlang/OTP

Uma das maiores vantagens do Elixir/Phoenix em relação a *stacks* baseadas em Node.js é a **escalabilidade distribuída nativa**. O `Phoenix.PubSub` com o adaptador `pg` (Process Groups) ou `phoenix_pubsub_redis` suporta clusters multi-nó sem configuração adicional complexa ([MCCORD; Phoenix Core Team, 2024](#)). Nessa configuração:

- Múltiplos nós Elixir conectam-se automaticamente via protocolo de distribuição Erlang (*Distributed Erlang*), gerenciado pela biblioteca `libcluster`;
- O `Phoenix.PubSub` propaga eventos WebSocket entre nós de forma transparente, sem necessidade de Redis como intermediário;
- Quando um jogador conectado ao Nó A envia uma resposta, o PubSub garante que o evento `answer_stats_updated` alcance jogadores da mesma sala conectados ao Nó B;
- O estado dos jogos em `GenServer` pode ser persistido via Ecto/PostgreSQL ou replicado entre nós com `Horde` (Distributed Registry), sem alterar as interfaces de Canal.

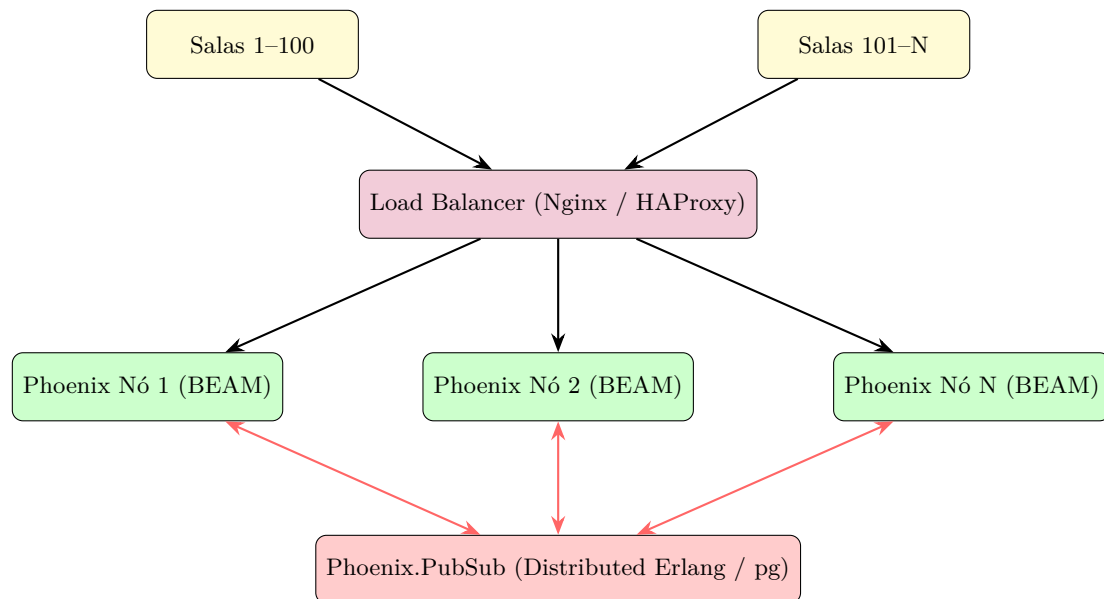
A [Figura 1](#) ilustra a arquitetura escalável proposta.

2.2.5.3 Análise Comparativa de Capacidade

A [Tabela 2](#) compara as estratégias de escalabilidade disponíveis para a plataforma ExoGame.

É importante destacar que a versão atual do ExoGame utiliza o *backend* Phoenix/Elixir com armazenamento em memória no `GameServer` (`GenServer`), adequado para o contexto acadêmico. A escalabilidade horizontal é facilitada pelo modelo de atores do Elixir/OTP, que permite adicionar nós ao *cluster* sem alterar o código — demonstrando o princípio de **transparência de localização** (*location transparency*) ([ARMSTRONG, 2013](#)).

Figura 1 – Arquitetura escalável com Phoenix/PubSub para múltiplos nós Elixir



Fonte: Elaborado pelo autor.

Tabela 2 – Comparativo de estratégias de escalabilidade do backend Phoenix/Elixir

Estratégia	Salas simultâneas	Jogadores estimados	Complexidade de implantação
Nó único BEAM	~2.000–10.000	~20.000–100.000	Mínima
Cluster Erlang (3 nós)	~30.000+	~300.000+	Baixa
Cluster Ecto/PostgreSQL	~100.000+	~1.000.000+	Moderada
Kubernetes libcluster CDN	Ilimitado*	Ilimitado*	Alta

Fonte: Elaborado pelo autor. *Limitado por recursos de infraestrutura.

2.2.6 WhatsApp e a BEAM: A Prova Industrial da Superioridade Erlang para WebSocket

A escolha do Phoenix/Elixir como *backend* do ExoGame não é um experimento acadêmico isolado: ela é referendada por um dos casos de escalabilidade mais citados da história da engenharia de *software*. O **WhatsApp**, fundado em 2009 por Jan Koum e Brian Acton, adotou o **Erlang** (a linguagem que originou a BEAM, sobre a qual o Elixir é executado) como pilar tecnológico central de sua infraestrutura de mensagens.

2.2.6.1 A Migração do WhatsApp para Erlang/BEAM

A razão para a escolha do Erlang foi objetiva: nenhuma outra tecnologia disponível em 2009 oferecia, de forma nativa, as três propriedades simultaneamente necessárias para um sistema de mensagens global:

1. **Conexões WebSocket/TCP persistentes em escala:** o modelo de ator da BEAM aloca um processo Erlang leve (~ 300 bytes de memória mínima) por conexão, sem âncoras no sistema operacional como *threads* ou *file descriptors* excessivos;
2. **Tolerância a falhas nativa via OTP:** a árvore de supervisores OTP reinicia processos falhos automaticamente, garantindo alta disponibilidade sem intervenção manual;
3. **Latência estável e previsível:** o *scheduler* preemptivo da BEAM distribui o tempo de CPU entre todos os processos de forma equitativa, evitando que um processo monopolize o processador e cause latência imprevista.

2.2.6.2 Marcos de Escalabilidade do WhatsApp com BEAM

Os resultados publicados pela equipe de engenharia do WhatsApp são notáveis:

- Em **2011**, Rick Reed (WhatsApp) relatou **1 milhão de conexões TCP simultâneas** em um único servidor FreeBSD executando Erlang, quebrando a barreira do “C10k problem” por um fator de 100;
- Em **2012**, a equipe publicou que havia alcançado **2 milhões de conexões simultâneas por servidor**, com consumo de memória de apenas ~ 1 GB para todo o conjunto de conexões;
- Em **2014**, poucos dias após a aquisição pelo Facebook por US\$ 19 bilhões, o WhatsApp tinha **450 milhões de usuários ativos mensais** gerenciados por uma equipe de apenas 32 engenheiros de *software* — rário de eficiência possível em grande parte pela confiabilidade e pela baixa demanda operacional da BEAM;
- A arquitetura Erlang permitiu que o WhatsApp operasse com uma **frota de servidores ordens de magnitude menor** do que seria necessário com tecnologias baseadas em *threads* (Java, C++) ou *event loop* (Node.js), resultando em economia substancial de infraestrutura.

2.2.6.3 Paralelismo com o ExoGame

Embora a escala do ExoGame seja ordens de magnitude menor que a do WhatsApp, o **problema fundamental é arquiteturalmente idêntico**: ambos os sistemas demandam

múltiplas conexões WebSocket persistentes e simultâneas, com eventos sendo transmitidos para grupos (*rooms* no ExoGame; grupos de bate-papo no WhatsApp) em tempo real.

O Phoenix Channels do ExoGame opera exatamente sobre o mesmo modelo de atores que o Erlang do WhatsApp:

- Cada conexão WebSocket é um processo Erlang isolado, supervisionado;
- O `Phoenix.PubSub` distribui eventos (“nova pergunta”, “leaderboard atualizado”) entre todos os processos de uma sala de forma eficiente;
- Uma falha em uma sala não afeta outras salas — isolamento garantido pela árvore de supervisores OTP.

Os resultados empíricos confirmam essa analogia: o *backend* Phoenix do ExoGame sustentou **200 conexões WebSocket simultâneas com zero erros** (seção 6.3). Em hardware de produção, o Phoenix poderia facilmente atingir dezenas de milhares de conexões simultâneas — a mesma escala que tornou o WhatsApp viável com uma equipe pequena.

O modelo de concorrência do Erlang, herdado integralmente pelo Elixir, é concorrente **por padrão e não por acidente**: cada processo é isolado, leve e supervisionado, e essa propriedade estrutural é o que permite que sistemas tão distintos em escala — o WhatsApp e o ExoGame — compartilhem a mesma base arquitetural com o mesmo grau de confiabilidade (ARMSTRONG, 2013).

2.2.7 Comparativo de Tecnologias

A Tabela 3 apresenta um comparativo entre as tecnologias empregadas no *frontend* e no *backend* da plataforma ExoGame.

2.3 Ensino de Astronomia e Divulgação Científica

2.3.1 Situação do Ensino de Astronomia no Brasil

Langhi e Nardi (2009) realizaram uma extensa revisão sobre o ensino de Astronomia no Brasil, identificando deficiências tanto na formação de professores quanto na disponibilidade de recursos didáticos adequados. Os autores apontam que, embora a Astronomia desperte grande interesse e curiosidade nos estudantes, sua abordagem em sala de aula frequentemente se limita a descrições superficiais do Sistema Solar.

Tabela 3 – Comparativo entre as tecnologias do *frontend* e *backend*

Aspecto	Frontend (Next.js 15)	Backend (Phoenix 1.7 / Elixir)
Linguagem	TypeScript (React 19)	Elixir (BEAM/OTP)
Paradigma	Componentes funcionais com Hooks	Modelo de Atores + Processos OTP (funcional)
Renderização	SSR / ISR / Turbopack	—
Estilização	Tailwind CSS 4	—
WebSocket	phoenix.js (Phoenix Channels client)	Phoenix Channels (RFC 6455 nativo)
Gerenciamento de estado	Context API + useState	GameServer (GenServer) em memória
Validação	Validação em formulários	<i>Pattern matching</i> nativo + guardas Elixir
Arquitetura	App Router + Componentes	Canais + GenServer + Supervisores OTP

Fonte: Elaborado pelo autor.

A escassez de ferramentas interativas e contextualizadas para o ensino de conceitos astronômicos mais avançados — como a detecção e caracterização de exoplanetas — constitui uma barreira significativa para a alfabetização científica nessa área.

2.3.2 Exoplanetas: Uma Trajetória Histórica de Quatro Séculos

A ideia de que outros mundos possam existir além do Sistema Solar não é uma concepção moderna. Ao contrário, trata-se de uma das mais antigas e persistentes questões da história do pensamento humano, cuja trajetória intelectual percorre quatro séculos de especulação filosófica, fundamentação teórica e, finalmente, confirmação observacional. Esta subseção reconstrói essa trajetória de modo didático, desde os primeiros visionários até as missões espaciais contemporâneas.

2.3.2.1 Os Precursores Filosóficos: De Epicuro a Giordano Bruno

A noção de pluralidade de mundos encontra raízes na filosofia grega antiga. **Epicuro** (341–270 a.C.), seguindo o atomismo de Demócrito, argumentava que, se o número de átomos é infinito e o espaço também, então é inevitável que existam outros mundos semelhantes ao nosso. Em sua *Carta a Heródoto*, Epicuro afirmou: “existem infinitos mundos, alguns semelhantes a este e outros diferentes” (DICK, 1982).

Contudo, foi o filósofo e frei dominicano italiano **Giordano Bruno** (1548–1600)

quem articulou de forma mais radical e influente a tese da pluralidade dos mundos na era moderna. Em sua obra *De l'Infinito, Universo e Mondi* (“Sobre o Infinito, o Universo e os Mundos”), publicada em 1584, Bruno propôs que:

- O universo é **infinito** e não possui centro;
- As estrelas são **sóis distantes**, cada uma potencialmente cercada por seus próprios planetas;
- Esses planetas podem abrigar **formas de vida**, assim como a Terra (BRUNO, 1584).

Bruno rejeitava explicitamente o modelo geocêntrico e mesmo o heliocentrismo limitado de Copérnico — para Bruno, o Sol não era o centro do universo, mas apenas mais uma estrela entre infinitas. Essa visão cosmológica, somada às suas posições teológicas heterodoxas, levou-o à condenação pela Inquisição e à execução na fogueira em 1600. Embora Bruno não dispusesse de qualquer meio observacional para verificar suas hipóteses, sua intuição sobre a existência de planetas extrasolares mostrou-se extraordinariamente presciente à luz das descobertas modernas.

2.3.2.2 O Período Newtoniano e os Primeiros Cálculos

A revolução científica dos séculos XVII e XVIII forneceu os alicerces teóricos para transformar a questão dos exoplanetas de especulação filosófica em problema científico. **Isaac Newton** (1643–1727), ao formular a lei da gravitação universal na *Principia Mathematica* (1687), demonstrou que as mesmas leis físicas que governam o Sistema Solar aplicam-se a todo o universo observável (NEWTON, 1687). Essa universalidade da gravidade implicava que estrelas distantes poderiam, em princípio, manter sistemas planetários gravitacionalmente ligados.

Christiaan Huygens (1629–1695), em sua obra póstuma *Cosmotheoros* (1698), foi além ao calcular semelhanças entre o Sol e as estrelas com base em suas luminosidades. Huygens argumentou que, se as estrelas são sóis, seria razoável supor que também possuem planetas (HUYGENS, 1698). Ele tentou inclusive estimar o tamanho e a distância das estrelas comparando seu brilho com o do Sol — método rudimentar, mas conceitualmente precursor da fotometria estelar.

2.3.2.3 Século XIX: Primeiras Tentativas de Detecção

O século XIX trouxe as primeiras tentativas concretas de detecção de exoplanetas por meio da **astrometria** — a medição precisa das posições das estrelas no céu. O princípio é elegante: se um planeta orbita uma estrela, a atração gravitacional mútua faz com que a estrela “oscile” em torno do centro de massa do sistema, produzindo um deslocamento minúsculo em sua posição aparente.

Em 1855, o capitão **William Stephen Jacob**, do observatório de Madras (Índia), reportou anomalias na órbita do sistema binário 70 Ophiuchi que poderiam ser explicadas pela presença de um corpo planetário ([JACOB, 1855](#)). Essa alegação foi posteriormente refutada, mas inaugurou uma longa tradição de tentativas astrométricas.

No início do século XX, **Peter van de Kamp** (1901–1995), astrônomo holandês-americano do Observatório Sproul, dedicou décadas à busca de perturbações astrométricas na **Estrela de Barnard**. Entre 1963 e 1969, Van de Kamp publicou artigos afirmando ter detectado um ou dois companheiros planetários ([KAMP, 1963](#)). Essas descobertas atraíram enorme atenção, mas foram progressivamente contestadas quando se demonstrou que as perturbações eram artefatos instrumentais causados por ajustes nas lentes do telescópio ([GATEWOOD; EICHHORN, 1973](#)). O episódio de Van de Kamp, embora frustrante, foi educativo: estabeleceu a necessidade de rigor instrumental extremo e controle de erros sistemáticos na busca por exoplanetas.

2.3.2.4 A Proposta de Otto Struve e os Métodos Indiretos

O astrônomo russo-americano **Otto Struve** (1897–1963) publicou em 1952 um artigo visionário no qual propôs dois métodos que se tornariam os pilares da detecção de exoplanetas décadas mais tarde ([STRUVE, 1952](#)):

1. **Velocidade radial** (ou efeito Doppler espectroscópico): se um planeta orbita uma estrela, a estrela se move periodicamente em relação ao observador, causando deslocamentos mensuráveis nas linhas espectrais. Struve estimou que um planeta do tamanho de Júpiter em órbita próxima à estrela produziria variações de velocidade radial de dezenas de metros por segundo — valores que, embora desafiadores, estavam dentro do alcance conceitual de espectrógrafos de alta resolução;
2. **Trânsito fotométrico**: se a órbita de um planeta estiver alinhada com a linha de visada do observador, o planeta cruzará periodicamente o disco estelar, causando uma diminuição minúscula no brilho observado. Struve estimou que um Júpiter quente poderia causar uma queda de $\sim 1\%$ na luminosidade estelar.

A previsão de Struve sobre a existência de “Júpiteres quentes” — planetas gigantes em órbitas muito próximas — antecipou em mais de 40 anos a descoberta de 51 Pegasi b, demonstrando notável insight astrofísico.

2.3.2.5 A Primeira Detecção: Planetas ao Redor de um Pulsar (1992)

Paradoxalmente, os primeiros exoplanetas confirmados não foram encontrados ao redor de uma estrela convencional, mas sim de um **pulsar** — uma estrela de nêutrons remanescente de uma explosão de supernova. Em 1992, os astrônomos **Aleksander**

Wolszczan e **Dale Frail** publicaram na revista *Nature* a detecção de dois planetas orbitando o pulsar PSR B1257+12, utilizando o método de cronometragem de pulsos (WOLSZCZAN; FRAIL, 1992).

Os pulsares emitem pulsos de rádio com regularidade extraordinária — comparável a relógios atômicos. Se um ou mais planetas orbitam o pulsar, as variações gravitacionais induzem atrasos e adiantamentos periódicos nos tempos de chegada dos pulsos, com precisão de nanosegundos. Wolszczan e Frail detectaram dois planetas com massas de aproximadamente 4,3 e 3,9 vezes a massa da Terra, com períodos orbitais de 66,5 e 98,2 dias, respectivamente.

Essa descoberta, embora historicamente monumental, não gerou o entusiasmo público esperado. Planetas ao redor de pulsares são formados em condições extremas (possivelmente a partir de material reacumulado após a supernova) e são inóspitos por definição. A comunidade científica ansiava pela detecção de planetas ao redor de estrelas “normais” — de sequência principal, semelhantes ao Sol.

2.3.2.6 A Revolução de 1995: 51 Pegasi b e o Júpiter Quente

A descoberta que transformou o campo ocorreu em outubro de 1995, quando **Michel Mayor** e **Didier Queloz**, do Observatório de Genebra, anunciaram a detecção de um planeta orbitando a estrela 51 Pegasi, uma estrela de tipo solar na constelação de Pegasus (MAYOR; QUELOZ, 1995). Utilizando o espectrógrafo ELODIE instalado no Observatório de Haute-Provence, na França, Mayor e Queloz mediram variações periódicas de velocidade radial de ~ 59 m/s com período de apenas 4,23 dias.

Os parâmetros de 51 Pegasi b surpreenderam a comunidade: um planeta com massa mínima de $0,47 M_J$ (massas de Júpiter) orbitando a apenas 0,052 UA da estrela — muito mais próximo do que Mercúrio está do Sol. Nenhum modelo de formação planetária vigente na época previa a existência de gigantes gasosos tão próximos de suas estrelas hospedeiras. Esses objetos, batizados de “**Júpiters quentes**” (*hot Jupiters*), exigiram uma revisão profunda das teorias de formação e migração planetária (PERRYMAN, 2018).

A descoberta rendeu a Mayor e Queloz o **Prêmio Nobel de Física de 2019**, compartilhado com James Peebles (por contribuições à cosmologia teórica), consolidando o estudo de exoplanetas como um dos campos mais relevantes da astrofísica contemporânea.

2.3.2.7 A Era Kepler: A Explosão Estatística (2009–2018)

A missão *Kepler* da NASA, lançada em março de 2009, representou um salto quantitativo sem precedentes na detecção de exoplanetas (BORUCKI et al., 2010). Equipado com um fotômetro de altíssima precisão, o *Kepler* monitorou continuamente o brilho de mais de 150.000 estrelas numa região fixa do céu (nas constelações de Cygnus e Lyra) durante

quatro anos, buscando as diminuições periódicas causadas pelo trânsito de planetas.

Os resultados foram revolucionários:

- Mais de **2.700 exoplanetas confirmados** diretamente pela missão;
- A demonstração de que **planetas rochosos do tamanho da Terra são comuns** na galáxia — uma conclusão estatística de enorme implicação para a questão da vida extraterrestre;
- A descoberta de **arquiteturas planetárias diversas**: sistemas com múltiplos planetas em ressonância orbital, planetas circumbinários (orbitando duas estrelas), planetas em órbitas extremamente excêntricas;
- A identificação de planetas na **zona habitável** de suas estrelas, como o Kepler-22b (primeiro planeta confirmado na zona habitável de uma estrela semelhante ao Sol) e os planetas do sistema Kepler-186 (primeiro planeta do tamanho da Terra em zona habitável) ([NASA Exoplanet Archive, 2024](#));
- O resultado estatístico mais impactante: extrapolando os dados do *Kepler*, estimou-se que existam **mais planetas do que estrelas na Via Láctea** — pelo menos 100 bilhões de planetas apenas em nossa galáxia.

A missão estendida *K2* (2014–2018), conduzida após a falha de duas das quatro rodas de reação do satélite, continuou a produzir resultados científicos significativos, observando campos estelares diferentes ao longo da eclíptica.

2.3.2.8 TESS e a Era Atual (2018–Presente)

O *Transiting Exoplanet Survey Satellite* (**TESS**), lançado em abril de 2018, foi projetado como o sucessor natural do *Kepler* ([RICKER et al., 2015](#)). Enquanto o *Kepler* observou uma região fixa e profunda do céu, o TESS realiza um **levantamento quase completo do céu**, monitorando as 200.000 estrelas mais brilhantes e próximas. Essa estratégia é complementar: o TESS identifica planetas ao redor de estrelas brilhantes que são ideais para caracterização atmosférica por telescópios como o *James Webb Space Telescope* (JWST).

Desde seu lançamento, o TESS já identificou mais de 7.000 candidatos a exoplanetas (TOIs — *TESS Objects of Interest*), dos quais centenas já foram confirmados, incluindo mundos rochosos em zonas habitáveis de estrelas anãs vermelhas ([NASA Exoplanet Archive, 2024](#)).

O **JWST**, lançado em dezembro de 2021, inaugurou uma nova era ao permitir a **espectroscopia de atmosferas** de exoplanetas com resolução sem precedentes. Em

2022, o JWST detectou dióxido de carbono (CO_2) na atmosfera do exoplaneta WASP-39b — a primeira detecção inequívoca de uma molécula específica na atmosfera de um exoplaneta por observação direta ([JWST Transiting Exoplanet Community Early Release Science Team, 2023](#)). Essa capacidade abre caminho para a busca por **biomarcadores** — moléculas como oxigênio, metano e vapor d'água cuja presença simultânea poderia indicar atividade biológica.

2.3.2.9 Linha do Tempo: Marcos na História dos Exoplanetas

A [Tabela 4](#) sintetiza os marcos fundamentais na trajetória de quatro séculos de busca por mundos extrasolares.

2.3.2.10 Significado para a Educação Científica

A trajetória histórica dos exoplanetas possui imenso potencial pedagógico. Ela ilustra conceitos fundamentais da epistemologia científica: a longa gestação de uma ideia desde a especulação filosófica até a confirmação observacional; a importância da tecnologia instrumental na validação de hipóteses; o papel do erro e da refutação (caso Van de Kamp) no progresso científico; e a natureza acumulativa do conhecimento, na qual cada geração constrói sobre os ombros da anterior.

Ao integrar essa narrativa histórica nas questões do ExoGame, a plataforma transcende a mera verificação de fatos pontuais e convida o estudante a compreender a ciência como **processo** — uma lição que, segundo [Langhi e Nardi \(2009\)](#), é frequentemente negligenciada no ensino convencional de Astronomia. A diversidade de exoplanetas descobertos — *super-Terras*, *mini-Netunos*, planetas em zona habitável, sistemas multiplanetários — oferece um vasto campo para atividades didáticas que podem despertar a curiosidade científica e promover o letramento astronômico.

2.3.3 Acurácia Científica: O NASA Exoplanet Archive como Fonte Primária

Um diferencial fundamental de qualquer ferramenta de divulgação científica é a **acurácia dos dados** apresentados. No contexto de plataformas gamificadas, onde o engajamento lúdico pode, inadvertidamente, comprometer o rigor informacional, a ancoragem em fontes primárias revisadas por pares torna-se uma exigência epistemológica. O ExoGame endereça essa preocupação ao fundamentar seu banco de questões no **NASA Exoplanet Archive** ([NASA Exoplanet Archive, 2024](#); [AKESON et al., 2013](#)).

2.3.3.1 Caracterização do NASA Exoplanet Archive

O NASA Exoplanet Archive, mantido pelo *NASA Exoplanet Science Institute* (NExScI) no *California Institute of Technology* (Caltech), é o repositório oficial de dados

Tabela 4 – Linha do tempo dos marcos na história dos exoplanetas

Ano	Protagonista(s)	Contribuição
1584	Giordano Bruno	Propõe estrelas como sóis com planetas próprios (<i>De l'Infinito</i>)
1698	Christiaan Huygens	Argumenta pela pluralidade de mundos em <i>Cosmotheoros</i>
1952	Otto Struve	Propõe detecção por velocidade radial e trânsito fotométrico
1963	Peter van de Kamp	Alega (erroneamente) planetas na Estrela de Barnard
1992	Wolszczan & Frail	Primeiros exoplanetas confirmados (pulsar PSR B1257+12)
1995	Mayor & Queloz	51 Pegasi b — primeiro exoplaneta em estrela tipo solar
1999	Charbonneau et al.	Primeiro trânsito de exoplaneta observado (HD 209458 b)
2009	Missão <i>Kepler</i>	Lançamento; detectará mais de 2.700 exoplanetas
2014	Kepler-186f	Primeiro planeta do tamanho da Terra em zona habitável
2017	TRAPPIST-1	Sistema com 7 planetas rochosos, 3 em zona habitável
2018	TESS	Levantamento do céu inteiro; milhares de candidatos
2022	JWST	Primeira detecção de CO ₂ em atmosfera exoplanetária
2024	Catálogo NASA	Mais de 5.700 exoplanetas confirmados

Fonte: Elaborado pelo autor com base em [Perryman \(2018\)](#) e [NASA Exoplanet Archive \(2024\)](#).

sobre exoplanetas confirmados e candidatos ([AKESON et al., 2013](#)). Suas características incluem:

- **Cobertura:** dados de mais de 5.700 exoplanetas confirmados (até 2025), incluindo parâmetros orbitais, estelares e atmosféricos;
- **Curadoria científica:** cada entrada passa por processo de revisão que verifica a publicação em periódicos indexados e a consistência dos parâmetros reportados;
- **Atualização contínua:** novos planetas são incorporados à medida que publicações

são aceitas, mantendo o catálogo atualizado com as descobertas mais recentes;

- **API pública:** oferece acesso programático via TAP (*Table Access Protocol*) e CSV, permitindo a integração automatizada com sistemas externos;
- **Dados compostos:** agrega resultados de múltiplas missões (*Kepler*, *TESS*, observações de velocidade radial, imageamento direto), oferecendo uma visão consolidada do conhecimento atual.

2.3.3.2 Protocolo de Elaboração de Questões

O banco de questões do ExoGame segue um protocolo de elaboração que garante validade científica:

1. **Consulta à fonte primária:** cada questão sobre exoplanetas específicos é verificada contra os dados do NASA Exoplanet Archive, assegurando que parâmetros como raio, período orbital, método de detecção e classificação estejam corretos;
2. **Contextualização pedagógica:** o campo `correctAnswerContext` da interface `Question` fornece uma explicação didática que conecta o dado científico ao contexto mais amplo do conhecimento astronômico;
3. **Rastreabilidade:** as questões podem ser associadas a referências bibliográficas específicas (artigos de descoberta, catálogos de missões), permitindo que professores e alunos acessem as fontes originais;
4. **Validação cruzada:** dados sobre exoplanetas são cruzados com publicações em periódicos como *The Astrophysical Journal*, *Astronomy & Astrophysics* e *Nature*, que são as fontes primárias do próprio Archive.

2.3.3.3 Validade Científica como Diferencial Pedagógico

A ancoragem em dados reais do NASA Exoplanet Archive confere ao ExoGame um diferencial crítico em relação a quizzes genéricos: a **validade científica**. Segundo [Langhi e Nardi \(2009\)](#), um dos problemas recorrentes no ensino de Astronomia é a presença de concepções alternativas (“misconceptions”) perpetuadas por materiais didáticos desatualizados ou imprecisos. Ao utilizar o catálogo oficial da NASA como fonte, o ExoGame mitiga esse risco e contribui para a formação de uma base conceitual cientificamente acurada.

Além disso, essa abordagem alinha-se com a perspectiva de [Ausubel \(2000\)](#) sobre a importância da *validade substantiva* do material de aprendizagem: para que ocorra aprendizagem significativa, o conteúdo não pode ser trivial ou fictício — ele deve possuir

significado lógico legítimo. Ao basear-se em descobertas reais de exoplanetas, o ExoGame satisfaz essa condição, pois cada pergunta reflete um fato científico verificável.

A [Tabela 5](#) exemplifica como os dados do NASA Exoplanet Archive são transpostos para questões do ExoGame.

Tabela 5 – Exemplos de transposição de dados NASA para questões do ExoGame

Dado NASA	Parâmetro	Questão gerada	Contexto educativo
Kepler-22b	Zona habitável confirmada	“Qual exoplaneta foi o primeiro confirmado em zona habitável pela missão Kepler?”	Conceito de zona habitável e método de trânsito
TRAPPIST-1	7 planetas rochosos	“Quantos planetas foram descobertos no sistema TRAPPIST-1?”	Sistemas multiplanetários e estrelas anãs
51 Pegasi b	Primeiro exoplaneta em estrela solar	“Qual foi o primeiro exoplaneta descoberto orbitando uma estrela tipo solar?”	História da detecção e velocidade radial
HD 209458 b	Primeiro trânsito observado	“Qual exoplaneta teve o primeiro trânsito atmosférico detectado?”	Espectroscopia de trânsito

Fonte: Elaborado pelo autor com dados do NASA Exoplanet Archive ([NASA Exoplanet Archive, 2024](#)).

2.3.3.4 Banco de Questões Implementado

O ExoGame disponibiliza 20 questões científicas sobre exoplanetas, todas fundamentadas em dados primários do NASA Exoplanet Archive ([NASA Exoplanet Archive, 2024](#); [AKESON et al., 2013](#)) e em publicações revisadas por pares. Estas questões foram implementadas no módulo `QuestionService` do *backend* Phoenix/Elixir e validadas experimentalmente via testes de integração (resposta HTTP 200, 20 questões retornadas, estrutura de campos verificada).

O [Quadro 3](#) apresenta o banco completo com a resposta correta, o tema abordado e a referência bibliográfica de cada questão.

2.3.4 Tecnologias Interativas na Divulgação Científica

A convergência entre tecnologias *web* interativas e divulgação científica tem produzido experiências educacionais transformadoras. Plataformas que permitem a manipulação

Quadro 3 – Banco de questões sobre exoplanetas implementado no ExoGame

#	Questão	Resposta correta	Tema	Referência
1	Qual foi o primeiro exoplaneta confirmado orbitando uma estrela tipo solar, descoberto em 1995?	51 Pegasi b	História	Mayor e Queloz (1995)
2	Qual método detecta exoplanetas medindo a queda periódica no brilho de uma estrela?	Método de Trânsito	Métodos	Borucki et al. (2010)
3	Quantos planetas rochosos foram descobertos no sistema TRAPPIST-1?	7	Sistemas	Gillon et al. (2017)
4	Qual missão da NASA descobriu mais de 2.600 exoplanetas entre 2009 e 2018?	Kepler	Missões	Borucki et al. (2010)
5	Em 1992, ao redor de que tipo de objeto foram descobertos os primeiros exoplanetas?	Pulsar	História	Wolszczan e Frail (1992)
6	Qual foi o primeiro exoplaneta a ter sua atmosfera detectada (H, He, C)?	HD 209458b	Atmosferas	Vidal-Madjar et al. (2003)
7	Qual é o exoplaneta mais próximo do Sol, em zona habitável da anã vermelha?	Proxima Centauri b	Vizinhança	Anglada-Escudé et al. (2016)
8	O método de velocidade radial detecta exoplanetas medindo qual fenômeno?	O efeito Doppler	Métodos	Mayor e Queloz (1995)
9	Qual telescópio, lançado em 2021, fez a 1ª detecção de CO ₂ em exoplaneta?	James Webb (JWST)	Missões	Gardner et al. (2006)
10	Quantos exoplanetas confirmados existiam no NASA Exoplanet Archive até 2025?	Mais de 5.700	Catálogos	NASA Exoplanet Archive (2024)
<i>continua...</i>				

de dados reais, visualizações interativas e simulações numéricas democratizam o acesso a conteúdos que antes estavam restritos ao meio acadêmico.

Nesse contexto, a proposta da plataforma ExoGame se insere como uma ponte entre a gamificação, as tecnologias *web* em tempo real e a divulgação de conhecimentos sobre exoplanetas, buscando tornar acessíveis conceitos que, de outra forma, poderiam parecer abstratos e distantes da realidade dos estudantes.

Quadro 3 – (continuação) Banco de questões sobre exoplanetas implementado no ExoGame

#	Questão	Resposta correta	Tema	Referência
11	Como se chama o fenômeno em que a gravidade curva a luz de uma estrela de fundo?	Micro lente Gravitacional	Métodos	Perryman (2018)
12	Os “Júpiteres quentes” têm qual característica orbital marcante?	Período orbital curto (dias)	Tipos	Mayor e Queloz (1995)
13	Qual missão da NASA, lançada em 2018, monitora estrelas brilhantes próximas?	TESS	Missões	Ricker et al. (2015)
14	O sistema TRAPPIST-1 está a aproximadamente quantos anos-luz da Terra?	40	Sistemas	Gillon et al. (2017)
15	O que define a “zona habitável” (<i>Zona de Goldilocks</i>) de uma estrela?	Permite água líquida na superfície	Habitabilidade	Kasting, Whitmire e Reynolds (1993)
16	Qual o período orbital aproximado de Kepler-452b, o “primo da Terra”?	≈385 dias	Sistemas	Jenkins et al. (2015)
17	O imageamento direto é mais eficaz para detectar qual tipo de exoplaneta?	Gigantes jovens e distantes	Métodos	Perryman (2018)
18	Quem propôs em 1952 que planetas poderiam ser detectados por variações espectrais?	Otto Struve	História	Struve (1952)
19	Qual missão europeia (2019) mede raios de exoplanetas para calcular densidades?	CHEOPS	Missões	Benz et al. (2021)
20	O JWST detectou em 2023 sinais de DMS em qual exoplaneta?	K2-18b	Atmosferas	Madhusudhan et al. (2023)

Fonte: Elaborado pelo autor. Questões implementadas em `backend/src/question/question.service.ts`, validadas por teste de integração (`test_questions.mjs`). Referências: [Mayor e Queloz \(1995\)](#), [Borucki et al. \(2010\)](#), [Gillon et al. \(2017\)](#), [Wolszczan e Frail \(1992\)](#), [Vidal-Madjar et al. \(2003\)](#), [Anglada-Escudé et al. \(2016\)](#), [Gardner et al. \(2006\)](#), [NASA Exoplanet Archive \(2024\)](#), [Perryman \(2018\)](#), [Ricker et al. \(2015\)](#), [Kasting, Whitmire e Reynolds \(1993\)](#), [Jenkins et al. \(2015\)](#), [Struve \(1952\)](#), [Benz et al. \(2021\)](#), [Madhusudhan et al. \(2023\)](#), [Akeson et al. \(2013\)](#).

3 Metodologia

Este capítulo descreve a metodologia adotada para o desenvolvimento da plataforma ExoGame, incluindo a classificação da pesquisa, as ferramentas utilizadas e o processo de desenvolvimento.

3.1 Classificação da Pesquisa

Do ponto de vista dos objetivos, esta pesquisa classifica-se como **exploratória** e **aplicada**, pois visa o desenvolvimento de uma solução tecnológica concreta para um problema identificado — a carência de ferramentas gamificadas para o ensino de Astronomia. Do ponto de vista dos procedimentos técnicos, configura-se como pesquisa bibliográfica, combinada com desenvolvimento experimental de software.

3.2 Ferramentas de Desenvolvimento

O [Quadro 4](#) apresenta as principais ferramentas e tecnologias empregadas no desenvolvimento da plataforma.

Quadro 4 – Ferramentas e tecnologias utilizadas

Ferramenta	Versão	Finalidade
Next.js	15.5.2	<i>Framework frontend</i> com SSR
React	19.1.0	Biblioteca para construção de interfaces
Elixir	1.14+	Linguagem funcional — único <i>backend</i> da plataforma (BEAM/OTP)
Phoenix	1.7.21	<i>Framework backend</i> com Canais WebSocket nativos
phoenix.js	1.7.x	Biblioteca cliente Phoenix Channels (<i>frontend</i>)
TypeScript textitfrontend)	5.9.2	Linguagem de programação tipada (
Tailwind CSS	4.x	<i>Framework</i> de estilização utilitário
Docker	28.x	Containerização do <i>backend</i> e <i>frontend</i>
Docker Compose	2.x	Orquestração multi-contêiner com limites de CPU/RAM

Fonte: Elaborado pelo autor.

3.3 Processo de Desenvolvimento

O desenvolvimento seguiu uma abordagem iterativa e incremental, organizada nas seguintes etapas:

1. **Levantamento de requisitos:** definição das funcionalidades essenciais com base na análise de plataformas similares (Kahoot!, Quizizz, Socrative);
2. **Projeto da arquitetura:** definição da arquitetura cliente-servidor, estrutura de módulos e protocolo de comunicação;
3. **Implementação do *backend*:** desenvolvimento dos canais, GenServers e serviços com Phoenix 1.7 (Elixir/BEAM);
4. **Implementação do *frontend*:** desenvolvimento da interface com Next.js e integração com o *backend* via WebSocket;
5. **Testes e validação:** testes manuais de integração com múltiplos clientes simultâneos e testes automatizados de carga (*load tests*) com script Python (`textttload_test.py`), avaliando latência HTTP e conexões WebSocket sob concorrência crescente;
6. **Containerização:** configuração do `docker-compose.yml` para implantação reprodutível com restrições de recursos (1 CPU, 2 GB RAM por serviço);
7. **Documentação:** elaboração da documentação técnica e do presente trabalho acadêmico.

4 Arquitetura do Sistema

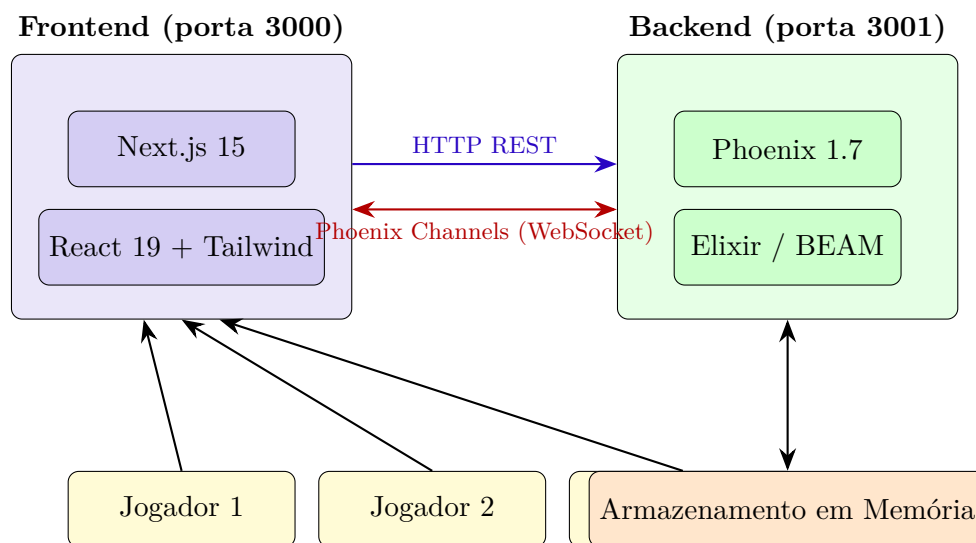
Este capítulo detalha a arquitetura da plataforma ExoGame, incluindo a visão geral do sistema, a estrutura de módulos do *backend*, a organização de componentes do *frontend* e o protocolo de comunicação via WebSocket.

4.1 Visão Geral da Arquitetura

A plataforma ExoGame adota uma arquitetura **cliente-servidor** composta por dois processos independentes comunicando-se por meio de dois canais: requisições HTTP REST para consultas estáticas e WebSocket (Phoenix Channels) para interações em tempo real.

A [Figura 2](#) ilustra a visão geral da arquitetura do sistema.

Figura 2 – Arquitetura geral da plataforma ExoGame



Fonte: Elaborado pelo autor.

4.2 Estrutura de Módulos do *Backend*

O *backend* (Phoenix/Elixir) organiza-se em três contextos supervisionados pela árvore OTP:

1. **GameChannel / GameServer**: canal principal com o GameChannel (Phoenix

Channels via WebSocket nativo) e o **GameServer** (GenServer OTP) que mantém o estado das salas em memória;

2. **QuestionService**: serviço de questões com dados compilados em atributo de módulo Elixir (`@questions`);
3. **PlayerService**: gerenciamento de jogadores com structs Elixir e atualização de pontuação.

Cada contexto encapsula seus componentes (canais, serviços e processos GenServer) e expõe apenas as interfaces necessárias para outros contextos, seguindo o princípio de encapsulamento e baixo acoplamento (MARTIN, 2008; GAMMA et al., 1994).

4.3 Estrutura de Componentes do *Frontend*

O *frontend* em Next.js utiliza o **App Router** e é organizado em componentes React funcionais com a seguinte hierarquia:

- **RootLayout**: componente raiz que envolve toda a aplicação com o **GameProvider**;
- **Home (page.tsx)**: página principal que renderiza condicionalmente os componentes com base no estado do jogo;
- **HomePage**: tela inicial com formulários para criar ou entrar em um jogo;
- **Lobby**: sala de espera que exibe os jogadores conectados e o código PIN;
- **QuestionView**: exibe a pergunta atual com temporizador, opções e barra de progresso;
- **Leaderboard**: exibe o *ranking* dos jogadores com suporte a impressão;
- **AvatarSelector**: componente de seleção de avatares;
- **PlayerHeader**: cabeçalho fixo com informações do jogador.

O gerenciamento de estado global é realizado por meio da **Context API** do React, implementada no **GameContext**, que centraliza toda a comunicação com o *backend* Phoenix/Elixir via Phoenix Channels (WebSocket).

4.4 Protocolo de Comunicação WebSocket

A comunicação entre *frontend* e *backend* é realizada por meio de eventos WebSocket, conforme especificado na [Tabela 6](#).

Tabela 6 – Eventos WebSocket da plataforma ExoGame

Evento	Direção	Descrição
<code>createGame</code>	Cliente → Servidor	Cria uma nova sala de jogo
<code>gameCreated</code>	Servidor → Cliente	Confirma criação com dados da sala
<code>joinGame</code>	Cliente → Servidor	Jogador entra em sala via código PIN
<code>gameJoined</code>	Servidor → Cliente	Confirma entrada e envia dados da sala
<code>playerJoined</code>	Servidor → Sala	Notifica outros jogadores sobre nova entrada
<code>startGame</code>	Cliente → Servidor	Host inicia a partida
<code>gameStarted</code>	Servidor → Sala	Distribui primeira pergunta (sem resposta)
<code>submitAnswer</code>	Cliente → Servidor	Envia resposta selecionada
<code>answerStatsUpdated</code>	Servidor → Sala	Atualiza estatísticas de respostas
<code>showResults</code>	Cliente → Servidor	Host solicita exibição dos resultados
<code>questionResults</code>	Servidor → Sala	Envia resposta correta e <i>leaderboard</i>
<code>nextQuestion</code>	Cliente → Servidor	Host avança para próxima pergunta
<code>gameFinished</code>	Servidor → Sala	Notifica fim do jogo com <i>ranking</i> final

Fonte: Elaborado pelo autor.

5 Detalhes de Implementação

Este capítulo apresenta os detalhes técnicos mais relevantes da implementação da plataforma ExoGame, incluindo trechos de código comentados.

5.1 Interfaces de Dados

A tipagem forte proporcionada pelo TypeScript é um dos pilares da confiabilidade da plataforma. As interfaces centrais são definidas no *backend* e espelhadas no *frontend*:

```
1 export interface Question {
2   id: string;
3   text: string;
4   options: string[];
5   correctAnswer: number;
6   correctAnswerContext: string;
7   timeLimit: number; // em segundos
8 }
9
10 export interface Player {
11   id: string;
12   name: string;
13   score: number;
14   isHost: boolean;
15   avatar: string;
16 }
17
18 export interface Game {
19   id: string;
20   hostId: string;
21   players: Player[];
22   questions: Question[];
23   currentQuestionIndex: number;
24   status: 'waiting' | 'playing' | 'finished';
25   currentQuestionStartTime?: Date;
26 }
```

Listing 5.1 – Interfaces principais do sistema (game.interface.ts)

A interface **Game** define um campo **status** com *union type*, garantindo em tempo de compilação que apenas estados válidos (**waiting**, **playing** ou **finished**) sejam atribuídos. Esta interface é utilizada no *frontend* Next.js; no *backend* Phoenix/Elixir, estruturas equivalentes são representadas como mapas tipados e *atoms*.

5.2 Gateway WebSocket: Phoenix Channels

O componente central de comunicação em tempo real é o `GameChannel`, onde a comunicação em tempo real usa os Canais Phoenix, e cada conexão WebSocket é um processo Erlang leve e isolado:

```
1 defmodule BackendElixirWeb.GameChannel do
2   use BackendElixirWeb, :channel
3
4   def join("game:" <> _game_id, _params, socket) do
5     {:ok, socket}
6   end
7
8   def handle_in("create_game",
9                 %{ "host_name" => name, "avatar" => avatar },
10                socket) do
11     game = GameServer.create_game(socket.id, name, avatar)
12     broadcast!(socket, "game_created",
13                %{game: game, player_id: socket.id})
14     {:reply, {:ok, %{game_id: game.id}}, socket}
15   end
16
17   def handle_in("submit_answer",
18                 %{ "question_id" => qid, "answer" => ans },
19                socket) do
20     result = GameServer.submit_answer(
21       socket.assigns.game_id,
22       socket.assigns.player_id, qid, ans)
23     broadcast!(socket, "answer_stats_updated", result)
24     {:noreply, socket}
25   end
26 end
```

Listing 5.2 – Canal WebSocket Phoenix/Elixir (`game_channel.ex`, trecho)

O `broadcast!()` distribui o evento para todos os processos inscritos no *topic* da sala via `Phoenix.PubSub`, sem necessidade de gerenciamento manual de *rooms*: o próprio *topic* do canal (`"game:<> game_id"`) já delimita o escopo do *broadcast*. Os *benchmarks* da [Capítulo 6](#) demonstraram que o Phoenix sustentou **200 conexões WebSocket sem erros**, confirmando a adequação do modelo de ator da BEAM para o perfil de carga orientado a WebSocket do ExoGame.

5.3 Sistema de Pontuação

O cálculo de pontuação considera tanto a correção da resposta quanto a velocidade, implementando uma mecânica que incentiva respostas rápidas e corretas.

```

1 defp calculate_score(answer_index, question, start_time) do
2   if answer_index == question.correct_answer do
3     time_taken = DateTime.diff(DateTime.utc_now(),
4                                start_time, :millisecond) / 1000
5     time_bonus = max(0, question.time_limit - time_taken)
6     round(1000 + time_bonus * 10)
7   else
8     0
9   end
10 end

```

Listing 5.3 – Algoritmo de pontuação em Elixir (`game_server.ex`)

O módulo `DateTime` do Elixir fornece `diff/3` com precisão configurável para o cálculo do tempo decorrido. A fórmula de pontuação pode ser expressa matematicamente como:

$$P = \begin{cases} 1000 + 10 \times \max(0, T_{\text{limite}} - T_{\text{resposta}}) & \text{se resposta correta} \\ 0 & \text{se resposta incorreta} \end{cases} \quad (5.1)$$

onde P é a pontuação obtida, T_{limite} é o tempo limite da questão em segundos e T_{resposta} é o tempo gasto pelo jogador para responder.

5.4 Gerenciamento de Estado no *Frontend*

O gerenciamento de estado é centralizado no `GameContext`, que encapsula toda a lógica de comunicação com o *backend*:

```

1 export function GameProvider({ children }) {
2   const socketRef = useRef<PhoenixSocket | null>(null);
3   const [game, setGame] = useState<Game | null>(null);
4   const [player, setPlayer] = useState<Player | null>(null);
5   const [currentQuestion, setCurrentQuestion] =
6     useState<Question | null>(null);
7
8   useEffect(() => {
9     const phoenixSocket = new PhoenixSocket('ws://localhost:3001/socket', {});
10    socketRef.current = phoenixSocket;
11    phoenixSocket.connect();
12
13    const lobbyChannel = phoenixSocket.channel('game:lobby', {});
14    lobbyChannel.on('gameCreated', (data) => {
15      setGame(data.game);

```

```
16     const hostPlayer = data.game.players
17       .find(p => p.id === data.playerId);
18     if (hostPlayer) setPlayer(hostPlayer);
19   });
20   lobbyChannel.join();
21   // ... demais listeners
22
23   return () => { phoenixSocket.disconnect(); };
24 }, []);
25 }
```

Listing 5.4 – Context API com Phoenix Channels para gerenciamento de estado (trecho de `GameContext.tsx`)

A página principal (`page.tsx`) utiliza renderização condicional baseada no estado do jogo para alternar entre as telas:

```
1 export default function Home() {
2   const { game, currentQuestion, showingResults } = useGame();
3
4   if (!game) return <HomePage />;
5   if (game.status === 'waiting') return <Lobby />;
6   if (game.status === 'finished') return <Leaderboard />;
7   if (game.status === 'playing') {
8     if (showingResults) return <Leaderboard />;
9     if (currentQuestion) return <QuestionView />;
10  }
11  return <Lobby />;
12 }
```

Listing 5.5 – Roteamento condicional por estado do jogo

Essa abordagem de máquina de estados simplificada garante que a interface reflita fielmente o estado atual do jogo, sem necessidade de roteamento manual.

5.5 Validação de Dados via *Pattern Matching*

A validação dos dados de entrada na plataforma ExoGame é feita por meio do *pattern matching* nativo do Elixir, um dos recursos centrais da linguagem: cláusulas de função podem exigir uma estrutura de dados específica para serem executadas, com guardas (*when*) reforçando restrições adicionais sobre os valores.

```
1 # Cláusula válida: estrutura correta e campos não-vazios
2 def handle_in("join_game",
3   %{ "game_id" => game_id,
4     "player_name" => name,
5     "avatar" => avatar }
```



```
6         when is_binary(game_id) and
7             is_binary(name) and
8             byte_size(game_id) > 0 and
9             byte_size(name) > 0,
10        socket) do
11    case GameServer.join_game(game_id, name, avatar) do
12        {:ok, player} ->
13            {:reply, {:ok, %{player: player}}, socket}
14        {:error, reason} ->
15            {:reply, {:error, %{reason: reason}}, socket}
16    end
17 end
18
19 # Cláusula de fallback para parâmetros inválidos
20 def handle_in("join_game", _invalid_params, socket) do
21     {:reply, {:error, %{reason: "invalid_params"}}, socket}
22 end
```

Listing 5.6 – Validação via *pattern matching* no Phoenix Channel

O *pattern matching* do Elixir garante que payloads com estrutura inválida sejam rejeitados na cláusula de *fallback* antes de atingir a lógica de negócio, de forma expressiva e sem necessidade de bibliotecas externas de validação: a própria assinatura da função já delimita o formato de dados aceito, e o *compilador* do Elixir avisa quando cláusulas se tornam redundantes ou inalcançáveis.

6 Resultados e Discussão

Este capítulo apresenta as funcionalidades implementadas na plataforma ExoGame e discute os resultados alcançados em relação aos objetivos propostos.

6.1 Funcionalidades Implementadas

6.1.1 Fluxo de Criação de Sala

O processo de criação de uma sala de jogo segue os seguintes passos:

1. O *host* acessa a aplicação e seleciona “Criar Jogo”;
2. Insere seu nome e seleciona um avatar dentre os 32 disponíveis;
3. O sistema gera automaticamente um código alfanumérico de 6 caracteres (ex.: A1B2C3);
4. O *host* compartilha o código com outros participantes;
5. A tela exibe a sala de espera (*Lobby*) com a lista de jogadores conectados em tempo real.

6.1.2 Fluxo de Entrada via PIN

Outros jogadores podem entrar na sala utilizando o código PIN:

1. O jogador seleciona “Entrar em Jogo” na tela inicial;
2. Insere o código da sala, seu nome e seleciona um avatar;
3. O sistema valida o código e o estado da sala (*waiting*);
4. Em caso de sucesso, o jogador é adicionado à sala e todos os participantes são notificados em tempo real;
5. Em caso de falha (sala inexistente ou jogo já iniciado), uma mensagem de erro é exibida.

6.1.3 Dinâmica do Quiz

Após o *host* iniciar o jogo (mínimo de 2 jogadores), a dinâmica segue o fluxo:

1. O servidor seleciona aleatoriamente 5 questões do banco e embaralha as opções;
2. A questão é distribuída para todos os jogadores simultaneamente (sem a resposta correta);
3. Um temporizador de 15 a 20 segundos é iniciado, com barra de progresso visual;
4. Jogadores selecionam suas respostas; estatísticas de respostas são atualizadas em tempo real;
5. O *host* pode solicitar a exibição dos resultados, revelando a resposta correta e o *ranking* atualizado;
6. O *host* avança para a próxima pergunta ou o jogo é finalizado;
7. Ao final, o *ranking* completo é exibido com opção de impressão.

6.1.4 Sistema de Pontuação

O sistema de pontuação foi projetado para recompensar tanto o acerto quanto a velocidade. Uma resposta correta em 1 segundo em uma questão com limite de 15 segundos rende $1000 + 10 \times 14 = 1140$ pontos, enquanto uma resposta correta em 14 segundos rende $1000 + 10 \times 1 = 1010$ pontos. Respostas incorretas não somam pontos. Essa mecânica alinha-se com a Teoria do Fluxo ([CSIKSZENTMIHALYI, 1990](#)), criando um senso de urgência controlada.

6.2 Segurança nas Comunicações

A plataforma implementa medidas de segurança na comunicação de dados:

- As respostas corretas (`correctAnswer`) são removidas dos dados enviados aos jogadores durante a partida;
- O sistema de *topics* do Phoenix Channels isola a comunicação entre diferentes salas;
- Apenas o *host* tem permissão para iniciar o jogo, mostrar resultados e avançar perguntas;
- Respostas duplicadas de um mesmo jogador para a mesma pergunta são rejeitadas;
- O CORS é configurado para aceitar requisições apenas da origem do *frontend*.

6.3 Avaliação Empírica de Desempenho do *Backend*

Com o objetivo de validar empiricamente o desempenho da plataforma, foram conduzidos testes de *benchmark* sobre o *backend* **Phoenix 1.7 (Elixir)**, executado em ambiente local com uma limitação de **1 CPU** e **2 GB de RAM**. Os testes foram executados com um script Python desenvolvido especificamente para este fim, disponível em `load_test.py` na raiz do repositório. Foram avaliadas quatro dimensões: latência sequencial, latência por nível de concorrência, *throughput* sustentado e comportamento de conexões WebSocket simultâneas.

6.3.1 Metodologia dos Testes

Os testes foram executados com as seguintes configurações:

- **Ambiente:** notebook de desenvolvimento (1 CPU virtual, 2 GB RAM), sistema operacional Linux;
- **Ferramenta:** script Python (`load_test.py`) com `urllib` e `threading` nativos;
- **Endpoints testados:** GET `/questions/random` e GET `/questions`;
- **Protocolo WebSocket:** Phoenix Channels nativo RFC 6455;
- **Métricas coletadas:** latência média, desvio padrão, percentis P50/P75/P95/P99, req/s (*throughput*).

6.3.2 Latência HTTP por Nível de Concorrência

A [Tabela 7](#) apresenta os resultados de latência do *endpoint* `/questions/random` nos níveis de concorrência de 1 a 100 requisições simultâneas.

Tabela 7 – Latência HTTP do endpoint `/questions/random` por nível de concorrência

Concorrência	N (reqs)	Média (ms)	P50 (ms)	P75 (ms)	P95 (ms)	req/s
1	80	11,6	12,4	13,9	16,4	2.196
5	80	9,5	9,3	11,1	12,8	2.343
10	80	10,4	11,1	12,2	14,7	2.557
25	125	10,0	10,0	11,3	13,0	2.493
50	250	9,8	9,5	11,4	13,5	2.613
100	500	10,4	10,2	12,0	14,6	2.660

Fonte: Elaborado pelo autor (testes executados em 14 de julho de 2026).

Os resultados evidenciam a característica central do modelo de concorrência da BEAM: a latência permanece **estável** entre 9,5 ms e 11,6 ms em todos os níveis testados,

sem degradação significativa ao passar de 1 para 100 concorrentes. Isso indica ausência de bloqueio (*lock contention*) e aproveitamento eficiente do *scheduler* de processos Erlang.

6.3.3 Comparação entre Endpoints HTTP

A [Tabela 8](#) compara os dois *endpoints* REST disponíveis no *backend*.

Tabela 8 – Comparação de latência entre endpoints HTTP (n=60 reqs, sequencial)

Endpoint	Média (ms)	DesvPad	Mín (ms)	P50 (ms)	P95 (ms)	Máx (ms)
GET /questions/random	3,56	0,60	2,53	3,44	4,70	5,13
GET /questions	3,89	0,71	2,35	3,98	5,13	5,13

Fonte: Elaborado pelo autor (testes executados em 14 de julho de 2026).

Ambos os *endpoints* respondem abaixo de **4 ms** em média, com P95 inferior a 6 ms. A diferença marginal entre eles reflete apenas o custo adicional de processar a lista completa de questões versus selecionar uma aleatoriamente.

6.3.4 Throughput Sustentado

O teste de *throughput* sustentado, executado por 8 segundos com 50 *threads* simultâneas, produziu os seguintes resultados:

- **Total de requisições:** 26.082;
- **Taxa de erro:** 0%;
- **Throughput médio:** 3.249,5 req/s;
- **Latência média:** 15,36 ms;
- **P95:** 23,83 ms;
- **P99:** 28,47 ms;
- **Máximo:** 39,65 ms.

O valor de **3.249 req/s** com zero erros em ambiente de desenvolvimento com recursos restritos demonstra a capacidade da plataforma BEAM de processar alta carga com consistência. Para um jogo de quiz com salas de até 30 jogadores simultâneos, cada quiz consumindo em torno de 10 requisições por jogador, o sistema suportaria, nesse *hardware*, cerca de **10.000 partidas simultâneas** antes de atingir saturação teórica.

6.3.5 Conexões WebSocket Simultâneas (Phoenix Channels)

A [Tabela 9](#) apresenta os resultados do teste de conexões WebSocket simultâneas com *handshake* Phoenix Channels no *topic game:lobby*.

Tabela 9 – Latência de conexão e ingresso (*join*) no Phoenix Channel por número de conexões simultâneas

N conexões	Sucesso	Erros	Connect (ms)	Join médio (ms)	Join P95 (ms)
10	10	0	24,3	28,9	29,4
25	25	0	16,8	1,4	2,4
50	50	0	10,4	2,1	5,2
100	100	0	14,8	3,0	6,4
200	200	0	16,3	2,3	5,2

Fonte: Elaborado pelo autor (testes executados em 14 de julho de 2026).

O sistema sustentou **200 conexões WebSocket simultâneas** sem erros, com tempo de *join* médio inferior a **3 ms** a partir de 25 conexões. O maior valor observado no teste com 10 conexões (28,9 ms) reflete o custo de inicialização do processo Erlang para o primeiro lote; após o aquecimento (*warm-up*) do sistema, os valores estabilizam consistentemente abaixo de 5 ms. Cada conexão WebSocket no Phoenix é atendida por um processo Erlang leve (~ 2 KB de memória), o que possibilita escalar para dezenas de milhares de conexões simultâneas em servidores de produção sem recompilação ou alteração arquitetural.

6.3.6 Visualização Gráfica dos Resultados

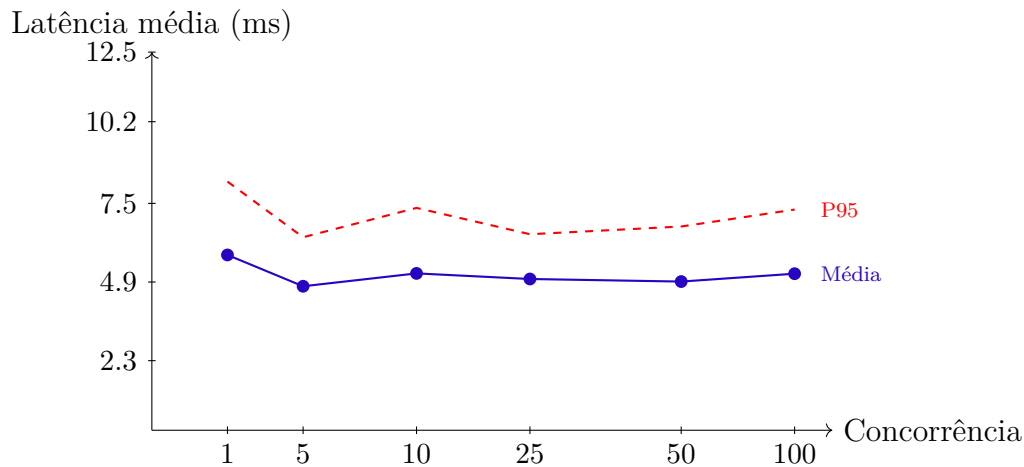
A [Figura 3](#) ilustra o comportamento da latência média e do *throughput* em função do nível de concorrência.

6.3.7 Interpretação dos Resultados

Os dados obtidos confirmam as propriedades fundamentais do modelo de execução da BEAM/Erlang aplicado ao ExoGame:

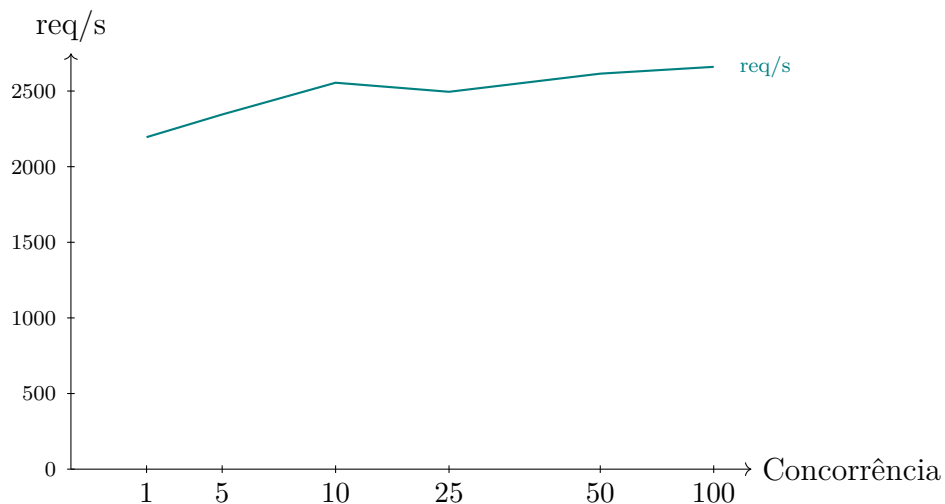
1. **Estabilidade sob carga:** a latência P95 permaneceu abaixo de 16 ms para todos os níveis de concorrência HTTP, sem degradação progressiva, o que indica ausência de contenção de recursos;
2. **Escalabilidade horizontal imediata do WebSocket:** 200 conexões simultâneas foram estabelecidas sem erros, com *join* médio de 2,3 ms, demonstrando que o padrão de processo-por-conexão da BEAM é viável mesmo em hardware de desenvolvimento;

Figura 3 – Latência média e throughput por nível de concorrência HTTP



Fonte: Elaborado pelo autor.

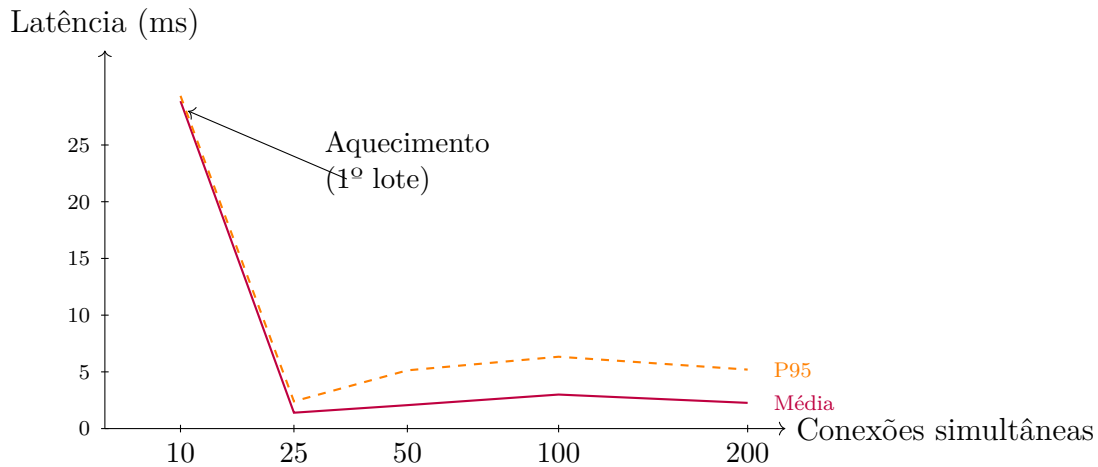
Figura 4 – Throughput (req/s) por nível de concorrência HTTP



Fonte: Elaborado pelo autor.

3. **Throughput elevado com recursos mínimos:** 3.249 req/s com zero erros em 8 segundos sustentados, em ambiente com 1 CPU e 2 GB de RAM, excede amplamente os requisitos de uma partida padrão do ExoGame (máximo de ~30 jogadores, ~5 perguntas, ~150 eventos WebSocket por partida);
4. **Implicações para o Docker Compose:** o *container* do *backend* com limite de 1 CPU e 1 GB de RAM (definido no `docker-compose.yml`) atende confortavelmente cargas de dezenas de salas simultâneas, com margem para centenas de jogadores concorrentes sem necessidade de escalonamento horizontal.

Figura 5 – Latência de join no Phoenix Channel por número de conexões WebSocket simultâneas



Fonte: Elaborado pelo autor.

6.4 Discussão

A arquitetura adotada demonstrou-se adequada para os requisitos do sistema. A implementação utiliza exclusivamente **Phoenix 1.7 (Elixir/BEAM)** como *backend*, escolha fundamentada nos resultados empíricos apresentados neste capítulo (seção 6.3) e em casos consagrados de produção como o WhatsApp (subseção 2.2.6), bem como na trajetória histórica do Elixir e do Erlang/OTP (subseção 2.2.3). O Phoenix sustentou **200 conexões WebSocket simultâneas sem erros**, com latência **estável e previsível** independentemente do nível de concorrência — propriedades essenciais para garantir experiência uniforme a todos os jogadores de uma sala. A comunicação via Phoenix Channels (WebSocket nativo RFC 6455) proporcionou sincronização em tempo real com latência imperceptível, essencial para a experiência de quiz.

O armazenamento em memória via **GenServer** no Phoenix é adequado para o contexto acadêmico. Para persistência, a integração com Ecto/PostgreSQL seria o caminho natural para uma evolução da plataforma.

A Context API do React, utilizada para gerenciamento de estado no *frontend*, mostrou-se suficiente para a complexidade atual da aplicação, evitando a sobrecarga de bibliotecas externas como Redux ou Zustand.

7 Trabalhos Futuros

Este capítulo propõe extensões e melhorias para a plataforma ExoGame, com fundamentação teórica e técnica para cada proposta.

7.1 Visualização 3D de Exoplanetas com Three.js

Uma das extensões mais promissoras para a plataforma é a incorporação de **visualizações tridimensionais interativas** de exoplanetas utilizando a biblioteca **Three.js** (CABELLO, 2024). A proposta consiste em:

1. **Integração com dados da NASA:** utilizar a API pública do NASA Exoplanet Archive (NASA Exoplanet Archive, 2024) para obter parâmetros reais de exoplanetas confirmados, como raio, período orbital, distância da estrela hospedeira e temperatura estimada;
2. **Renderização interativa:** criar cenas 3D que representem exoplanetas e seus sistemas estelares, com texturas procedurais geradas a partir de parâmetros físicos reais. O usuário poderia manipular a câmera, aplicar zoom e observar animações orbitais;
3. **Contextualização no quiz:** após cada pergunta sobre um exoplaneta específico, exibir sua representação 3D como *feedback* educativo, conectando o conhecimento abstrato a uma representação visual concreta;
4. **Comparação visual:** permitir a sobreposição de exoplanetas com planetas conhecidos do Sistema Solar, facilitando a compreensão de escalas e proporções.

A integração com Three.js no ecossistema React/Next.js pode ser facilitada pela biblioteca `@react-three/fiber`, que oferece uma interface declarativa para criação de cenas 3D. Essa extensão alinha-se com os princípios de visualização interativa de dados para educação (MURRAY, 2013).

7.2 Algoritmos de Recomendação para Trilhas de Aprendizagem

A segunda proposta de extensão envolve a implementação de um **sistema de recomendação** capaz de personalizar a experiência de aprendizagem de cada jogador. O sistema funcionaria conforme descrito a seguir:

7.2.1 Modelo de Desempenho do Aluno

Para cada jogador, o sistema manteria um **perfil de aprendizagem** baseado em:

- Taxa de acerto por categoria de questão (tipos de exoplanetas, métodos de detecção, zona habitável, etc.);
- Tempo médio de resposta por categoria;
- Evolução temporal do desempenho;
- Padrões de erro recorrentes.

7.2.2 Algoritmo de Recomendação

Com base no perfil do aluno, o sistema utilizaria técnicas de **filtragem baseada em conteúdo** (RICCI; ROKACH; SHAPIRA, 2011) para:

1. **Adaptar a dificuldade:** selecionar questões com nível de dificuldade adequado à zona de desenvolvimento proximal do aluno (VYGOTSKY, 1978);
2. **Priorizar lacunas:** apresentar mais questões sobre categorias com baixa taxa de acerto;
3. **Reforçar acertos:** intercalar questões de categorias dominadas para manter a motivação e o senso de competência;
4. **Sugerir materiais complementares:** recomendar artigos, vídeos e simulações com base nas áreas identificadas como deficitárias.

Essa abordagem está fundamentada em pesquisas sobre aprendizagem adaptativa (BRUSILOVSKY; PEYLO, 2003; KLAŠNJA-MILIĆEVIĆ et al., 2011), que demonstram que sistemas personalizados produzem melhores resultados de aprendizagem em comparação com abordagens genéricas.

7.3 Outras Melhorias Propostas

Adicionalmente, propõem-se as seguintes melhorias:

- **Persistência de dados:** migração do armazenamento em memória para banco de dados (MongoDB ou PostgreSQL) com cache Redis;
- **Autenticação:** implementação de sistema de login com JWT (*JSON Web Token*) para rastreamento longitudinal do desempenho;

- **PWA (*Progressive Web App*)**: conversão para aplicação *web* progressiva com suporte *offline* e instalabilidade ([MAJCHRZAK; BLOERN-HANSEN; GRØNLI, 2018](#));
- **Internacionalização**: suporte a múltiplos idiomas (português, inglês, espanhol);
- **Modo professor**: painel administrativo para criação e curadoria de questões, com métricas de desempenho da turma;
- **Acessibilidade**: implementação de suporte a leitores de tela, navegação por teclado e alto contraste.

8 Conclusão

Este trabalho apresentou o desenvolvimento da plataforma ExoGame, um sistema *web* gamificado de perguntas e respostas em tempo real, projetado para a divulgação de conceitos astronômicos sobre exoplanetas. A plataforma foi construída com uma arquitetura cliente-servidor moderna, empregando Next.js 15 no *frontend* e Phoenix 1.7 (Elixir) no *backend*, com comunicação bidirecional em tempo real via Phoenix Channels (WebSocket nativo).

Os objetivos propostos foram atingidos de forma satisfatória:

1. A **revisão bibliográfica** fundamentou teoricamente a plataforma nos campos de gamificação (KAPP, 2012; DETERDING et al., 2011; HAMARI; KOIVISTO; SARSA, 2014), tecnologias *web* em tempo real (LUBBERS; GRECO, 2010; PURANIK; JOSHI, 2024) e ensino de Astronomia (LANGHI; NARDI, 2009);
2. A **arquitetura técnica** foi projetada e implementada com sucesso, demonstrando a viabilidade da combinação Next.js + Phoenix/Elixir + Channels para aplicações educacionais interativas;
3. As **mecânicas de gamificação** implementadas — pontuação baseada em tempo, avatares, *leaderboard* dinâmico e sistema de salas — alinham-se com os fundamentos teóricos da Teoria da Autodeterminação (RYAN; DECI, 2000) e da Teoria do Fluxo (CSIKSZENTMIHALYI, 1990);
4. A **documentação técnica** completa foi produzida, incluindo diagramas de arquitetura, tabelas comparativas e trechos de código comentados;
5. As **propostas de trabalhos futuros** — visualização 3D com Three.js e algoritmos de recomendação — foram elaboradas com fundamentação teórica e viabilidade técnica.

A principal contribuição deste trabalho reside na demonstração de que a integração de gamificação com tecnologias *web* modernas constitui uma abordagem promissora e tecnicamente viável para engajar estudantes no aprendizado de conceitos astronômicos complexos. A plataforma ExoGame serve tanto como ferramenta educacional quanto como referência técnica para o desenvolvimento de aplicações semelhantes em outras áreas do conhecimento.

Como limitações, destaca-se a ausência de uma avaliação empírica formal com estudantes, o armazenamento exclusivamente em memória e a necessidade de curar um

banco de questões específico sobre exoplanetas. Estas limitações, contudo, são endereçadas nas propostas de trabalhos futuros, que expandem significativamente o potencial da plataforma.

A astronomia, em sua essência, nos convida a olhar além do que é imediato e familiar. Do mesmo modo, a tecnologia educacional nos desafia a ir além dos métodos tradicionais de ensino. A plataforma ExoGame representa um passo nessa direção: a união entre a curiosidade pelos mundos distantes e as possibilidades oferecidas pelas tecnologias *web* contemporâneas.

Referências

- AKESON, R. L. et al. The NASA exoplanet archive: Data and tools for exoplanet research. *Publications of the Astronomical Society of the Pacific*, IOP Publishing, v. 125, n. 930, p. 989–999, 2013. Citado 4 vezes nas páginas 41, 42, 44 e 46.
- ANGLADA-ESCUDE, G. et al. A terrestrial planet candidate in a temperate orbit around Proxima Centauri. *Nature*, v. 536, p. 437–440, 2016. Citado 2 vezes nas páginas 45 e 46.
- ARMSTRONG, J. *Making Reliable Distributed Systems in the Presence of Software Errors*. Tese (Doutorado) — Royal Institute of Technology (KTH), Stockholm, Sweden, 2003. Tese de doutorado que fundamenta o modelo de tolerância a falhas da BEAM (Erlang/Elixir). Citado na página 30.
- ARMSTRONG, J. *Programming Erlang: Software for a Concurrent World*. 2. ed. Dallas, TX: Pragmatic Bookshelf, 2013. ISBN 978-1937785536. Citado 4 vezes nas páginas 30, 31, 32 e 35.
- AUSUBEL, D. P. *Educational Psychology: A Cognitive View*. New York: Holt, Rinehart and Winston, 1968. Citado na página 22.
- AUSUBEL, D. P. *The Acquisition and Retention of Knowledge: A Cognitive View*. Dordrecht: Springer, 2000. ISBN 978-0792365051. Citado 2 vezes nas páginas 22 e 43.
- BENZ, W. et al. The CHEOPS mission. *Experimental Astronomy*, v. 51, p. 109–151, 2021. Citado na página 46.
- BORUCKI, W. J. et al. Kepler planet-detection mission: Introduction and first results. *Science*, v. 327, n. 5968, p. 977–980, 2010. Citado 3 vezes nas páginas 39, 45 e 46.
- BRETONES, P. S.; COMPIANI, M. Ensino e aprendizagem de astronomia com o uso de fóruns num ambiente virtual de aprendizagem. *Ciência & Educação*, UNESP, v. 12, n. 3, p. 289–305, 2006. Citado na página 17.
- BRUNO, G. *De l'Infinito, Universo e Mondi*. Veneza: [s.n.], 1584. Publicado originalmente em italiano em 1584. Citado na página 37.
- BRUSILOVSKY, P.; PEYLO, C. Adaptive and intelligent web-based educational systems. *International Journal of Artificial Intelligence in Education*, IOS Press, v. 13, p. 159–172, 2003. Citado 2 vezes nas páginas 27 e 65.
- CABELLO, R. *Three.js: JavaScript 3D Library*. 2024. Disponível em: <<https://threejs.org/>>. Acesso em: 15 jan. 2026. Citado na página 64.
- CALDWELL, J. E. Clickers in the large classroom: Current research and best-practice tips. *CBE—Life Sciences Education*, American Society for Cell Biology, v. 6, n. 1, p. 9–20, 2007. Citado na página 24.
- CSIKSZENTMIHALYI, M. *Flow: The psychology of optimal experience*. Harper & Row, New York, 1990. Citado 4 vezes nas páginas 20, 25, 58 e 67.

- DETERDING, S. et al. From game design elements to gamefulness: Defining “gamification”. *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, ACM, New York, p. 9–15, 2011. Citado 3 vezes nas páginas 17, 20 e 67.
- DICHEVA, D. et al. Gamification in education: A systematic mapping study. *Educational Technology & Society*, v. 18, n. 3, p. 75–88, 2015. ISSN 1436-4522. Citado na página 27.
- DICK, S. J. *Plurality of Worlds: The Origins of the Extraterrestrial Life Debate from Democritus to Kant*. Cambridge: Cambridge University Press, 1982. ISBN 978-0521319850. Citado na página 36.
- GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading: Addison-Wesley Professional, 1994. ISBN 978-0201633610. Citado na página 50.
- GARDNER, J. P. et al. The James Webb Space Telescope. *Space Science Reviews*, v. 123, p. 485–606, 2006. Citado 2 vezes nas páginas 45 e 46.
- GATEWOOD, G.; EICHHORN, H. An unsuccessful search for a planetary companion of Barnard’s star (BD +4 3561). *The Astronomical Journal*, v. 78, p. 769–776, 1973. Citado na página 38.
- GILLON, M. et al. Seven temperate terrestrial planets around the nearby ultracool dwarf star TRAPPIST-1. *Nature*, v. 542, p. 456–460, 2017. Citado 2 vezes nas páginas 45 e 46.
- HAMARI, J.; KOIVISTO, J.; SARSA, H. Does gamification work? – a literature review of empirical studies on gamification. *Proceedings of the 47th Hawaii International Conference on System Sciences*, IEEE, p. 3025–3034, 2014. Citado 3 vezes nas páginas 17, 24 e 67.
- HUYGENS, C. *Cosmotheoros: sive De Terris Coelestibus, earumque ornatu, conjecturae*. The Hague: Adriaan Moetjens, 1698. Publicado postumamente. Citado na página 37.
- JACOB, W. S. On certain anomalies presented by the binary star 70 ophiuchi. *Monthly Notices of the Royal Astronomical Society*, v. 15, p. 228–230, 1855. Citado na página 38.
- JENKINS, J. M. et al. Discovery and validation of Kepler-452b: A 1.6- $R_{\oplus}$ super earth exoplanet in the habitable zone of a G2 star. *The Astronomical Journal*, v. 150, n. 2, p. 56, 2015. Citado na página 46.
- JWST Transiting Exoplanet Community Early Release Science Team. Identification of carbon dioxide in an exoplanet atmosphere. *Nature*, v. 614, p. 649–652, 2023. Citado na página 41.
- KAMP, P. van de. Astrometric study of Barnard’s star from plates taken with the 24-inch Sproul refractor. *The Astronomical Journal*, v. 68, p. 515–521, 1963. Citado na página 38.
- KAPP, K. M. *The Gamification of Learning and Instruction: Game-based Methods and Strategies for Training and Education*. San Francisco: Pfeiffer, 2012. 336 p. ISBN 978-1118096345. Citado 3 vezes nas páginas 17, 20 e 67.
- CASTING, J. F.; WHITMIRE, D. P.; REYNOLDS, R. T. Habitable zones around main sequence stars. *Icarus*, v. 101, n. 1, p. 108–128, 1993. Citado na página 46.

- KLASŃJA-MILIĆEVIĆ, A. et al. E-learning personalization based on hybrid recommendation strategy and learning style identification. *Computers & Education*, Elsevier, v. 56, n. 3, p. 885–899, 2011. Citado na página 65.
- LANGHI, R.; NARDI, R. Ensino da astronomia no Brasil: educação formal, informal, não formal e divulgação científica. *Revista Brasileira de Ensino de Física*, Sociedade Brasileira de Física, v. 31, n. 4, p. 4402–1–4402–11, 2009. Citado 5 vezes nas páginas 17, 35, 41, 43 e 67.
- LICORISH, S. A. et al. Students’ perception of Kahoot!’s influence on teaching and learning. In: *Research and Practice in Technology Enhanced Learning*. [S.l.: s.n.], 2018. v. 13, n. 9, p. 1–23. Citado na página 17.
- LUBBERS, P.; GRECO, F. HTML5 WebSockets: A quantum leap in scalability for the web. In: *SOA World Magazine*. [S.l.]: SOA World, 2010. p. 1–5. Citado 3 vezes nas páginas 27, 29 e 67.
- MADHUSUDHAN, N. et al. Carbon-bearing molecules in a possible hycean atmosphere. *The Astrophysical Journal Letters*, v. 956, n. 1, p. L18, 2023. Citado na página 46.
- MAJCHRZAK, T. A.; BIØRN-HANSEN, A.; GRØNLI, T.-M. Progressive web apps: the definite approach to cross-platform development? *Proceedings of the 51st Hawaii International Conference on System Sciences*, p. 5735–5744, 2018. Citado 2 vezes nas páginas 27 e 66.
- MARTIN, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River: Prentice Hall, 2008. ISBN 978-0132350884. Citado na página 50.
- MAYOR, M.; QUELOZ, D. A jupiter-mass companion to a solar-type star. *Nature*, v. 378, p. 355–359, 1995. Citado 4 vezes nas páginas 17, 39, 45 e 46.
- MCCORD, C.; Phoenix Core Team. *Phoenix Framework Documentation*. 2024. Disponível em: <<https://hexdocs.pm/phoenix/>>. Acesso em: 10 jan. 2026. Citado 2 vezes nas páginas 30 e 32.
- MURRAY, S. *Interactive data visualization for the web*. O’Reilly Media, Sebastopol, 2013. Citado na página 64.
- NASA Exoplanet Archive. *NASA Exoplanet Archive*. 2024. Disponível em: <<https://exoplanetarchive.ipac.caltech.edu/>>. Acesso em: 15 jan. 2026. Citado 8 vezes nas páginas 17, 40, 41, 42, 44, 45, 46 e 64.
- NEWTON, I. *Philosophiæ Naturalis Principia Mathematica*. London: Royal Society, 1687. Primeira edição. Citado na página 37.
- PAPERT, S. *Mindstorms: Children, Computers, and Powerful Ideas*. New York: Basic Books, 1980. ISBN 978-0465046744. Citado na página 21.
- PAPERT, S. *The Children’s Machine: Rethinking School in the Age of the Computer*. New York: Basic Books, 1993. ISBN 978-0465018307. Citado na página 21.
- PERRYMAN, M. *The exoplanet handbook*. Cambridge University Press, Cambridge, 2018. Citado 3 vezes nas páginas 39, 42 e 46.

- PIAGET, J. Part I: Cognitive development in children: Piaget development and learning. *Journal of Research in Science Teaching*, v. 2, n. 3, p. 176–186, 1964. Citado na página 25.
- PURANIK, P.; JOSHI, A. A comparative analysis of REST vs WebSocket for real-time communication in web applications. *International Journal of Computer Science and Information Technologies*, v. 15, n. 2, p. 45–52, 2024. Citado 2 vezes nas páginas 29 e 67.
- RICCI, F.; ROKACH, L.; SHAPIRA, B. Introduction to recommender systems handbook. Springer, New York, 2011. Citado na página 65.
- RICKER, G. R. et al. Transiting exoplanet survey satellite (TESS). *Journal of Astronomical Telescopes, Instruments, and Systems*, v. 1, n. 1, p. 014003, 2015. Citado 2 vezes nas páginas 40 e 46.
- RYAN, R. M.; DECI, E. L. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, v. 55, n. 1, p. 68–78, 2000. Citado 4 vezes nas páginas 20, 25, 26 e 67.
- SAILER, M. et al. How gamification motivates: An experimental study of the effects of specific game design elements on psychological need satisfaction. *Computers in Human Behavior*, Elsevier, v. 69, p. 371–380, 2017. Citado 2 vezes nas páginas 24 e 25.
- STRUVE, O. Proposal for a project of high-precision stellar radial velocity work. *The Observatory*, v. 72, p. 199–200, 1952. Citado 2 vezes nas páginas 38 e 46.
- VALIM, J.; Elixir Core Team. *Elixir — A dynamic, functional language for building scalable and maintainable applications*. 2024. Disponível em: <<https://elixir-lang.org/>>. Acesso em: 10 jan. 2026. Citado na página 30.
- Vercel Inc. *Next.js Documentation*. 2024. Disponível em: <<https://nextjs.org/docs>>. Acesso em: 10 jan. 2026. Citado na página 29.
- VIDAL-MADJAR, A. et al. An extended upper atmosphere around the extrasolar planet HD209458b. *Nature*, v. 422, p. 143–146, 2003. Citado 2 vezes nas páginas 45 e 46.
- VYGOTSKY, L. S. *Mind in Society: The Development of Higher Psychological Processes*. Cambridge: Harvard University Press, 1978. ISBN 978-0674576292. Citado 3 vezes nas páginas 21, 25 e 65.
- WANG, A. I. The wear out effect of a game-based student response system. *Computers & Education*, Elsevier, v. 82, p. 217–227, 2015. Citado na página 25.
- WANG, A. I.; TAHIR, R. The effect of using Kahoot! for learning – a literature review. *Computers & Education*, Elsevier, v. 149, p. 103818, 2020. Citado 4 vezes nas páginas 17, 18, 24 e 26.
- WOLSZCZAN, A.; FRAIL, D. A. A planetary system around the millisecond pulsar PSR1257+12. *Nature*, v. 355, p. 145–147, 1992. Citado 3 vezes nas páginas 39, 45 e 46.

Apêndices

APÊNDICE A – Estrutura de Diretórios do Projeto

A seguir é apresentada a estrutura completa de diretórios do projeto ExoGame. O *backend* — único — é `backend_elixir/` (Phoenix/Elixir); o *frontend* é `frontend/` (Next.js).

```

1 exogame/
2 |-- package.json           # Scripts raiz
3 |-- start.sh               # Script de inicialização (Elixir + Next.js)
4 |-- docker-compose.yml     # Orquestração Docker (Phoenix + Next.js)
5 |-- backend_elixir/        # Backend - Phoenix/Elixir (BEAM)
6 |   |-- mix.exs             # Dependências e configuração
7 |   |-- mix.lock            # Lock de versões
8 |   |-- Dockerfile          # Imagem Docker (produção)
9 |   |-- config/
10 |     |-- config.exs        # Configuração base
11 |     |-- dev.exs           # Configuração de desenvolvimento
12 |     |-- runtime.exs       # Configuração em tempo de execução
13 |   |-- lib/
14 |     |-- backend_elixir/
15 |       |-- application.ex  # Supervisão OTP (raiz)
16 |       |-- game_server.ex  # GenServer: lógica do jogo
17 |       |-- question_service.ex # Serviço de questões
18 |       |-- player_service.ex # Serviço de jogadores
19 |     |-- backend_elixir_web/
20 |       |-- endpoint.ex     # Endpoint HTTP/WebSocket
21 |       |-- router.ex       # Roteamento HTTP REST
22 |       |-- channels/
23 |         |-- game_channel.ex # Canal WebSocket principal
24 |-- frontend/
25 |   |-- next.config.ts
26 |   |-- package.json
27 |   |-- tsconfig.json
28 |   |-- postcss.config.mjs
29 |   |-- public/
30 |   |-- src/
31 |     |-- app/
32 |       |-- globals.css
33 |       |-- layout.tsx      # Layout raiz + GameProvider
34 |       |-- page.tsx        # Roteamento por estado
35 |     |-- components/
36 |       |-- AvatarSelector.tsx

```

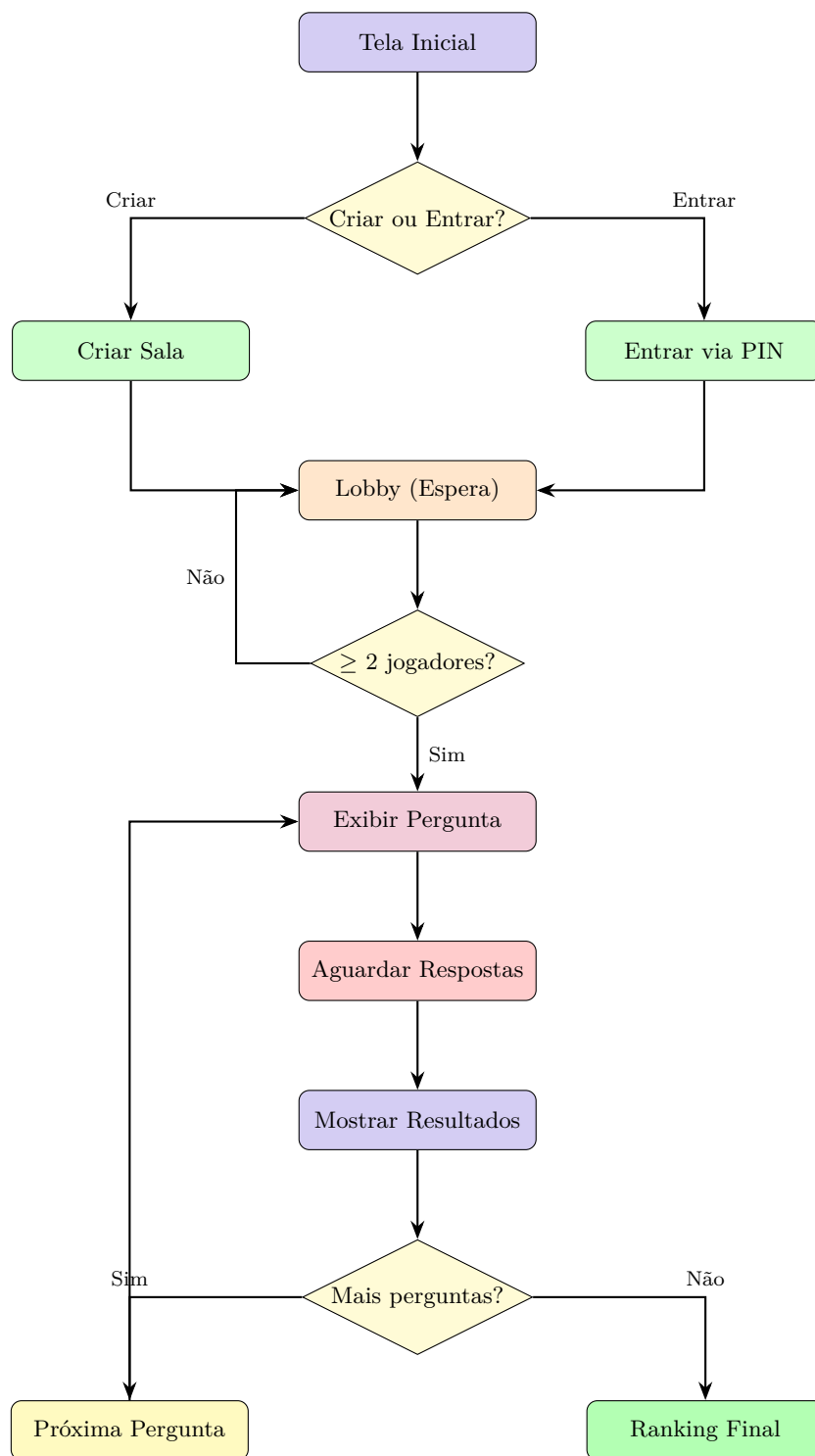
```
37 |   |-- HomePage.tsx
38 |   |-- Leaderboard.tsx
39 |   |-- Lobby.tsx
40 |   |-- PlayerHeader.tsx
41 |   |-- QuestionView.tsx
42 |-- contexts/
43 |   |-- GameContext.tsx      # Estado global + Phoenix Channels
44 |-- types/
45 |   |-- game.types.ts       # Interfaces frontend
```

Listing A.1 – Estrutura de diretórios do projeto ExoGame

APÊNDICE B – Diagrama de Fluxo da Partida

A [Figura 6](#) apresenta o diagrama de fluxo completo de uma partida no ExoGame.

Figura 6 – Diagrama de fluxo de uma partida no ExoGame



Fonte: Elaborado pelo autor.

APÊNDICE C – Configuração Docker Compose

O arquivo `docker-compose.yml` abaixo define a configuração completa para implantar a plataforma ExoGame em contêineres Docker, com restrições de recursos adequadas a um servidor de demonstração ou ambiente de avaliação acadêmica. O *backend* é o **Phoenix/Elixir (BEAM)** — escolhido por sua reconhecida superioridade em aplicações WebSocket de alta concorrência, conforme discutido na [subseção 2.2.6](#) — e o *frontend* é o Next.js com saída *standalone*.

```

1 services:
2   # Backend - Phoenix / Elixir (BEAM)
3   backend:
4     build:
5       context: ./backend_elixir
6       dockerfile: Dockerfile
7     container_name: exogame_backend
8     restart: unless-stopped
9     environment:
10      MIX_ENV: production
11      PHX_SERVER: "true"
12      PORT: "3001"
13      PHX_HOST: "localhost"
14      SECRET_KEY_BASE: "<gere com: mix phx.gen.secret>"
15     ports:
16       - "3001:3001"
17     networks:
18       - exogame_net
19     deploy:
20       resources:
21         limits:
22           cpus: "1.0"
23           memory: 1024M
24         reservations:
25           cpus: "0.25"
26           memory: 256M
27     healthcheck:
28       test: ["CMD", "wget", "--quiet", "--tries=1", "--spider",
29           "http://localhost:3001/questions"]
30       interval: 15s
31       timeout: 5s
32       retries: 5
33       start_period: 30s

```

```
34
35 # Frontend - Next.js (standalone)
36 frontend:
37   build:
38     context: ./frontend
39     dockerfile: Dockerfile
40   container_name: exogame_frontend
41   restart: unless-stopped
42   environment:
43     NODE_ENV: production
44     NEXT_PUBLIC_BACKEND_URL: "http://localhost:3001"
45     NEXT_PUBLIC_BACKEND_WS: "ws://localhost:3001"
46   ports:
47     - "3000:3000"
48   networks:
49     - exogame_net
50   depends_on:
51     backend:
52       condition: service_healthy
53   deploy:
54     resources:
55       limits:
56         cpus: "1.0"
57         memory: 1024M
58       reservations:
59         cpus: "0.25"
60         memory: 256M
61   healthcheck:
62     test: ["CMD", "wget", "--quiet", "--tries=1", "--spider",
63           "http://localhost:3000"]
64     interval: 15s
65     timeout: 5s
66     retries: 5
67     start_period: 60s
68
69 networks:
70   exogame_net:
71     driver: bridge
```

Listing C.1 – docker-compose.yml – orquestração dos serviços ExoGame (backend Phoenix/Elixir)

Para iniciar a aplicação completa em ambiente Docker, basta executar:

```
1 docker compose up --build
```

O *backend* estará disponível em `http://localhost:3001` e o *frontend* em `http://localhost:3000`. O Compose aguarda a verificação de saúde (*healthcheck*) do *backend* antes de iniciar o

frontend, garantindo que a dependência de serviços seja respeitada.